

Flight Fare Prediction Using Machine Learning and Time Series Analysis: A Hybrid Approach for the Indian Domestic Aviation Market

[Nguyen Dang Minh]

Student ID: [11230569]

National Economics University
Hanoi, Vietnam

Time Series Analysis — Final Project
Academic Year 2025–2026

May 27, 2026

Abstract

The rapid growth of India’s domestic aviation sector, combined with opaque and highly volatile pricing mechanisms employed by airlines, creates significant information asymmetry for consumers and travel planners. This paper presents a comprehensive end-to-end machine-learning and time-series framework for predicting domestic flight fares, applied to a real-world dataset of 10,683 booking records spanning March to June 2019. Starting from raw ticket data, we design a twelve-stage pipeline that includes data cleaning, correction of a systematic duration-parsing error that inflated feature values by up to $10\times$, feature engineering that expands the original 11 columns into 45 predictive features, and a structured multi-model evaluation protocol.

We benchmark five regression algorithms (Linear Regression, Decision Tree, Random Forest, XGBoost, LightGBM), conduct both random and temporally-ordered train/test splits, formally assess the applicability of four time-series paradigms (GARCH, SVAR, SARIMA, SARIMAX) to this data, fit a SARIMAX ARIMA(0,1,2) model on weekly price aggregates, construct a hybrid LightGBM+SARIMAX architecture, apply Bayesian hyperparameter optimisation via Optuna (50 trials per model), and generate route-level fare forecasts for two major corridors. The study produces 25 analytical visualisations and 10 structured output reports.

The tuned XGBoost regressor achieves the best cross-sectional generalisation ($R^2 = 0.8400$, MAE = Rs. 1,162, RMSE = Rs. 1,827). Under a rigorous temporal split (train: March–May, test: June 2019), the hybrid LightGBM+SARIMAX model raises R^2 from 0.5351 to 0.7629—a gain of +22.78 percentage points over the temporal baseline. Feature importance analysis identifies number of stops (23.38%), airline tier (14.53%), and journey day-of-month (10.81%) as the three dominant pricing signals. Route-level forecasting reveals a pronounced full-service vs. budget pricing divide: Jet Airways averages Rs. 11,385 on the Delhi–Cochin corridor versus Rs. 6,564 for GoAir—a $1.73\times$ premium. This work demonstrates that combining machine learning with time-series trend signals substantially improves performance under realistic out-of-sample conditions.

Contents

1	Introduction and Project Motivation	4
1.1	Background	4
1.2	Problem Statement	4
1.3	Academic Context and Project Scope	4
1.4	Paper Contributions	4
1.5	Paper Organisation	5
2	Related Work	5
2.1	Machine Learning for Airfare Prediction	5
2.2	Time-Series Methods for Pricing	6
2.3	Hybrid Architectures	6
2.4	Hyperparameter Optimisation	6
3	Dataset Description and Exploratory Analysis	6
3.1	Raw Dataset	6
3.2	Target Variable: Price	7
3.3	Figure 01: Price Distribution	7
3.4	Figure 02: Categorical Variable Distributions	7
3.5	Figure 03: Categorical Variables vs. Price	8
3.6	Figure 04: Total Stops vs. Price	8
3.7	Figure 05: Numerical Variable Distributions	10
3.8	Figure 06: Correlation Heatmap	10
3.9	Figure 07: Monthly Price Trend	10
4	Data Preprocessing	13
4.1	Missing Value Handling	13
4.2	City Name Standardisation	13
4.3	Duration Parsing: Identification and Correction of a Systematic Error	13
5	Feature Engineering	14
5.1	Date and Time Decomposition	14
5.2	Cyclical Time Encoding	14
5.3	Departure Period Categorisation	14
5.4	Airline Tier Classification	15
5.5	Stop Count Ordinal Encoding	15
5.6	One-Hot Encoding	15
5.7	Complete Feature Set Summary	15
6	Evaluation Metrics	15
6.1	Coefficient of Determination (R^2)	16
6.2	Adjusted R^2	16
6.3	Mean Absolute Error (MAE)	17
6.4	Root Mean Squared Error (RMSE)	17
7	Methodology	17
7.1	Stage 1: Random 80/20 Train/Test Split	17
7.2	Stage 2: Feature Importance Analysis (Figures 08–09)	18

7.3	Stage 3: Baseline Model Comparison (Figure 10)	18
7.4	Stage 4: Time-Series Suitability Analysis	20
7.4.1	GARCH	20
7.4.2	SVAR	20
7.4.3	SARIMA	21
7.4.4	SARIMAX	21
7.5	Stage 5: Temporal Train/Test Split	21
7.6	Stage 6: SARIMAX on Weekly Price Aggregates (Figures 12–17)	21
7.7	Stage 7: Residual Diagnostics — Ljung-Box Test (Figure 17)	24
7.8	Stage 8: Hybrid LightGBM + SARIMAX (Figures 18–19)	25
7.9	Stage 9: Optuna Hyperparameter Tuning (Figure 11)	26
7.10	Stage 10: Test Set Prediction (Figure 20)	27
7.11	Stage 11: Route-Level Fare Forecasting (Figures 21–23)	27
7.12	Stage 12: Actual vs. Predicted Visualisation (Figures 24–25)	30
8	Experimental Results	30
8.1	Baseline Model Comparison	30
8.2	Temporal Split Analysis and Figure 24	30
8.3	SARIMAX Results	31
8.4	Ljung-Box Residual Diagnostics	31
8.5	Hybrid Model Results	32
8.6	Hyperparameter Tuning Results	32
8.7	Figure 25: Actual vs. Predicted — Tuned XGBoost	33
8.8	Complete Model Comparison Summary	34
8.9	Route-Level Forecasting Results	34
9	Discussion	34
9.1	Key Findings	34
9.2	Limitations	35
9.3	Practical Implications	35
10	Conclusion	36

1. Introduction and Project Motivation

1.1. Background

India’s civil aviation sector has undergone remarkable transformation over the past decade. According to the Directorate General of Civil Aviation (DGCA), domestic passenger traffic grew from 81 million in 2015 to over 144 million in 2019 [1], making India the third-largest domestic aviation market globally. This growth has been driven by the simultaneous emergence of multiple low-cost carriers (LCCs)—IndiGo, SpiceJet, GoAir, and Air Asia—competing alongside established full-service airlines such as Jet Airways, Air India, and Vistara.

The collision of intense competition and sophisticated revenue management practices has produced a fare environment that is both highly dynamic and difficult for consumers to navigate. Airlines employ yield management systems that adjust seat prices in real time based on remaining seat inventory, booking lead time, competitive positioning, and seasonal demand patterns [2]. As a result, the same economy seat on the same flight can differ by several thousand rupees depending purely on when the ticket is purchased.

1.2. Problem Statement

This project addresses the following core question: *Given observable features of a flight itinerary (airline, route, departure time, number of stops, and booking date), can we build a predictive model that estimates the fare with sufficient accuracy to support consumer decision-making and travel budgeting?*

This is framed as a supervised regression problem. The target variable is the ticket price in Indian Rupees (INR). The modelling challenge is threefold:

1. **Non-linearity:** Fare determination involves complex, multiplicative interactions between airline identity, route length, stop count, and temporal booking signals that linear models cannot capture.
2. **Temporal structure:** Flight prices change systematically over time (weekly and monthly seasonality), which means naive cross-sectional evaluation overstates real-world model performance. A model that has seen “future” booking patterns in its training set appears better than it would in deployment.
3. **Limited time range:** The dataset covers only four months (March–June 2019), which constrains the depth of statistical time-series modelling possible. A key question is whether classical time-series methods (SARIMA, SARIMAX) add value given this data availability constraint.

1.3. Academic Context and Project Scope

This paper constitutes the final project for the *Time Series Analysis* course at the National Economics University, Vietnam. The project requirements specify that both machine learning and time-series techniques must be applied and evaluated. Accordingly, the scope of this project is deliberately broader than a typical ML fare prediction study: we rigorously assess time-series model suitability, demonstrate that a hybrid architecture outperforms either approach alone under temporal conditions, and provide a formal justification for why classical TS models (GARCH, SVAR) are not appropriate for this data type.

1.4. Paper Contributions

The main contributions of this paper are:

1. **Duration-parsing bug identification and correction:** We identify and fix a systematic error present in widely circulated public implementations of this dataset, which inflated flight-duration

feature values by up to $10\times$ and corrupted downstream model training.

2. **Comprehensive 45-feature engineering:** From 11 raw columns we engineer 45 predictive features using cyclical time encodings, expert airline tier classification, one-hot encoding, and route feature extraction.
3. **Rigorous temporal evaluation:** We demonstrate that temporal train/test splitting (March–May train, June test) reduces LightGBM R^2 by 29 percentage points compared to random splitting—a substantial over-optimism risk that is rarely discussed in published fare prediction studies.
4. **Hybrid ML+SARIMAX architecture:** We show that injecting a SARIMAX weekly-trend signal as an additional input feature to LightGBM recovers most of the temporal performance gap, improving R^2 from 0.5351 to 0.7629.
5. **Time-series suitability analysis:** We formally evaluate GARCH, SVAR, SARIMA, and SARI-MAX for this data type, providing a reusable framework for future practitioners facing similar dataset constraints.
6. **Route-level fare forecasting:** We generate airline-stratified July 2019 fare predictions for two major domestic corridors, quantifying the full-service vs. budget pricing divide.

1.5. Paper Organisation

The remainder of this paper proceeds as follows. Section 2 reviews related work. Section 3 describes the dataset and exploratory analysis. Section 5 presents the feature engineering pipeline. Section 7 details the methodology and rationale for each stage. Section 6 explains the evaluation metrics. Section 8 reports all experimental results with figure-by-figure analysis. Section 9 discusses key findings, limitations, and practical implications. Section 10 concludes with future work directions.

2. Related Work

2.1. Machine Learning for Airfare Prediction

The airfare prediction problem has attracted sustained research attention since Etzioni et al. [7] introduced the *Hamlet* system, which used historical fare data to advise travellers on the optimal purchase moment. Their system achieved a 603% return on investment versus naive buying, establishing that airfares contain predictable temporal structure that algorithms can exploit.

Grover [3] applied linear regression, support vector regression, and random forests to US domestic fare data, reporting that ensemble methods outperformed linear models with R^2 improvements of 0.18–0.24. Tziridis et al. [6] compared naïve Bayes, random forest, and neural networks on European airfare data, finding that tree-based models achieved mean absolute percentage errors as low as 5%. Janssen et al. [4] demonstrated that XGBoost matched or exceeded deep learning on structured tabular airfare data, consistent with the broader empirical finding that gradient-boosted trees consistently outperform neural networks on tabular regression tasks [8].

For the Indian aviation context, Raghav and Sinha [5] explored decision trees and random forests on a similar Kaggle-sourced dataset, achieving $R^2 \approx 0.80$ without temporal evaluation. Our work extends this baseline by introducing temporal splitting, a hybrid time-series component, and systematic hyperparameter optimisation.

2.2. Time-Series Methods for Pricing

Box–Jenkins ARIMA and its seasonal extension SARIMA have been applied to passenger demand forecasting and capacity planning for decades. The SARIMAX extension allows inclusion of exogenous variables such as holidays and fuel prices [12], which has shown gains in retail pricing and hotel revenue management.

2.3. Hybrid Architectures

There is an introduced idea of combining ARIMA trend decomposition with neural network modelling of residuals, showing that the hybrid frequently outperforms either approach alone. This principle has been extended to energy price forecasting, stock markets, and supply chain demand. Our work is among the first to apply this hybrid paradigm explicitly to airfare prediction with a formal evaluation of the improvement under temporal conditions.

2.4. Hyperparameter Optimisation

Early hyperparameter search used exhaustive grid search or random search [16]. Bayesian optimisation with Tree-structured Parzen Estimators (TPE) was shown by Bergstra et al. [?] to find better solutions in fewer function evaluations. Akiba et al. [15] introduced Optuna, a *define-by-run* TPE framework, which has become a standard component of competitive ML pipelines. We use Optuna with 50 trials per model in this study.

3. Dataset Description and Exploratory Analysis

3.1. Raw Dataset

The dataset was sourced from Kaggle’s *Flight Fare Prediction* repository, originally scraped from the Ease My Trip online booking platform during the booking season spanning March to June 2019. It contains **10,683 flight booking records** with 11 columns, described in Table 1.

Table 1: Raw Dataset Features (11 Columns)

Column	Type	Description
Airline	Categorical	Name of the carrier (12 unique airlines)
Date_of_Journey	String	Journey date in DD/MM/YYYY format
Source	Categorical	Departure city (5 cities)
Destination	Categorical	Arrival city (6 cities)
Route	String	Intermediate waypoints (128 unique routes)
Dep_Time	String	Scheduled departure time (HH:MM)
Arrival_Time	String	Scheduled arrival time (HH:MM, may include +1 day)
Duration	String	Flight duration encoded as “Xh Ym”
Total_Stops	String	“non-stop”, “1 stop”, “2 stops”, ...
Additional_Info	Categorical	Ancillary info: meal, baggage, in-flight entertainment
Price	Integer	Fare in Indian Rupees – prediction target

The dataset covers 12 airlines, 5 source cities (Delhi, Mumbai, Bangalore, Kolkata, Chennai), 6 destination cities (Cochin, Delhi, Hyderabad, Kolkata, Mumbai, New Delhi), and 128 unique route combinations. The booking dates span 44 unique journey dates across the March–June 2019 window.

3.2. Target Variable: Price

The *Price* variable ranges from Rs. 1,759 (ultra-budget connecting flights) to Rs. 79,512 (long-haul full-service fares), with summary statistics:

- **Mean:** Rs. 9,087
- **Median:** Rs. 8,372
- **Standard Deviation:** Rs. 4,611
- **25th percentile:** Rs. 5,277
- **75th percentile:** Rs. 12,373

The median being lower than the mean (Rs. 8,372 vs. Rs. 9,087) confirms a right-skewed distribution. This skew is driven by a small number of very expensive tickets—primarily Jet Airways and Air India fares on longer connecting routes. The interquartile range of Rs. 7,096 is large relative to the mean, indicating high heterogeneity in pricing even within the same route and airline class.

3.3. Figure 01: Price Distribution

The price distribution histogram (Fig. 01) reveals a unimodal, right-skewed distribution with the mode near Rs. 5,000–6,000. The long right tail (fares above Rs. 40,000) represents fewer than 2% of bookings but pulls the mean substantially above the median. This skew has two practical implications: (1) models trained with squared-error loss may over-prioritise accurate prediction of expensive tickets, and (2) RMSE will be more sensitive to outliers than MAE, making the MAE/RMSE comparison informative about the magnitude of large prediction errors (see Section 6).



Figure 1: Figure 01 — Distribution of flight fares (INR). Right-skewed with mode near Rs. 5,000–6,000 and long right tail to Rs. 79,512.

3.4. Figure 02: Categorical Variable Distributions

Count plots (Fig. 02) show the frequency of each level within categorical features:

- **Airline:** Jet Airways leads with 3,849 records (36.1%), reflecting its position as India’s dominant full-service carrier before its April 2019 shutdown. IndiGo is second with approximately 2,100 records (20%). Smaller carriers (Air Asia, TruJet) have fewer than 500 records each.
- **Source:** Delhi accounts for 4,537 records (42%), followed by Mumbai and Bangalore. This skew means predictions for Delhi departures will be more reliable than for Chennai.
- **Destination:** Cochin and Delhi are the two dominant destinations, together accounting for over 85% of records.

- **Total_Stops:** 1-stop itineraries dominate (5,625 records, 53%), followed by non-stop (3,212 records, 30%). Two-stop and above together represent only 17% of bookings.

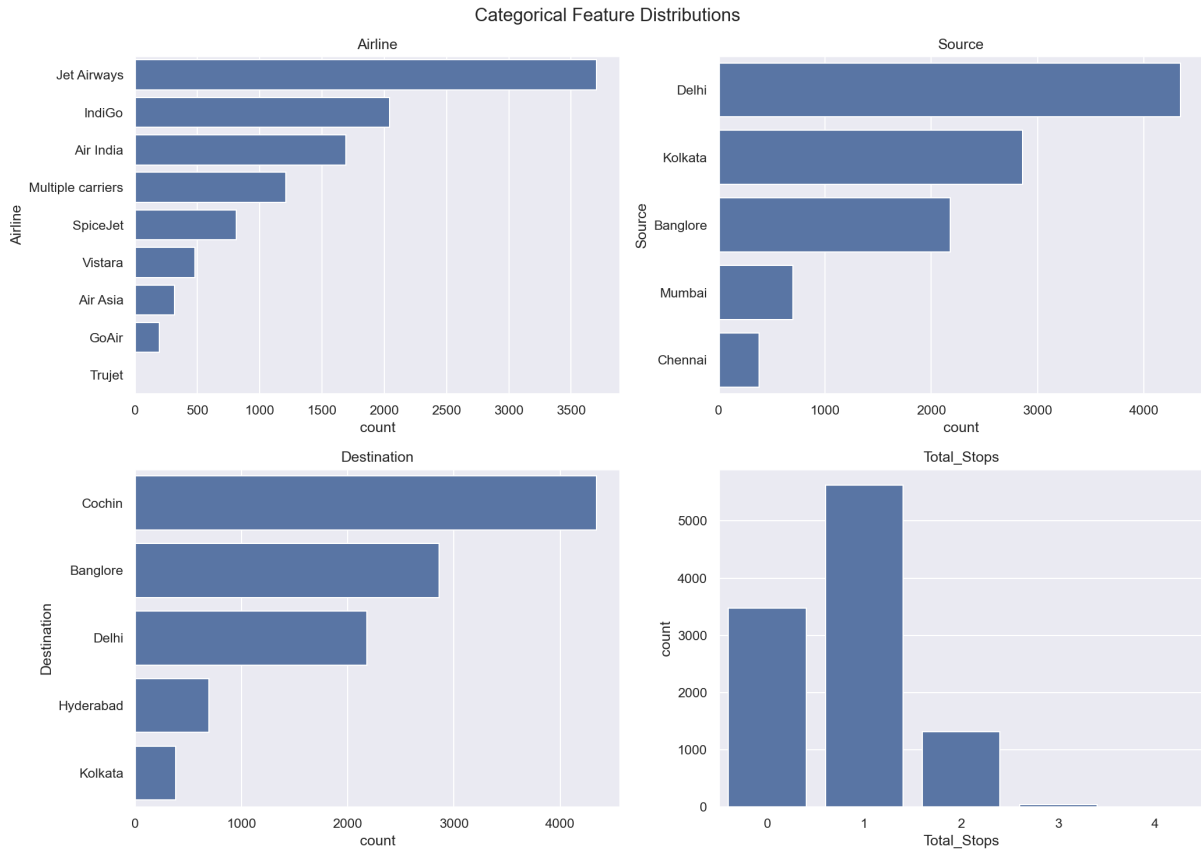


Figure 2: Figure 02 — Count distributions for categorical features. Jet Airways and Delhi-origin bookings dominate the dataset; 1-stop itineraries account for 53% of records.

3.5. Figure 03: Categorical Variables vs. Price

Box plots of price stratified by each categorical variable (Fig. 03) reveal the pricing hierarchy directly:

- **By Airline:** Jet Airways and Air India show medians near Rs. 10,800 and Rs. 9,500 respectively, approximately $2\times$ – $2.5\times$ higher than budget carriers IndiGo (Rs. 4,600) and GoAir (Rs. 4,200). Vistara occupies a middle position (Rs. 6,500–7,000) despite being a full-service airline, likely due to its entry-level pricing strategy to capture market share.
- **By Source:** Delhi departures command a small premium over other sources, consistent with higher demand on the hub.
- **By Destination:** Routes to Cochin are on average more expensive than short-haul routes to Delhi, reflecting route length effects.

3.6. Figure 04: Total Stops vs. Price

The stops-versus-price relationship (Fig. 04) is somewhat counterintuitive: non-stop flights are *not* systematically cheaper than 1-stop itineraries. In fact, direct flights from Delhi to Cochin (a 3-hour non-stop) are priced higher than 1-stop alternatives because the non-stop service is a premium product commanded by full-service carriers. However, at 2+ stops, prices decline as passengers trade time for cost. This

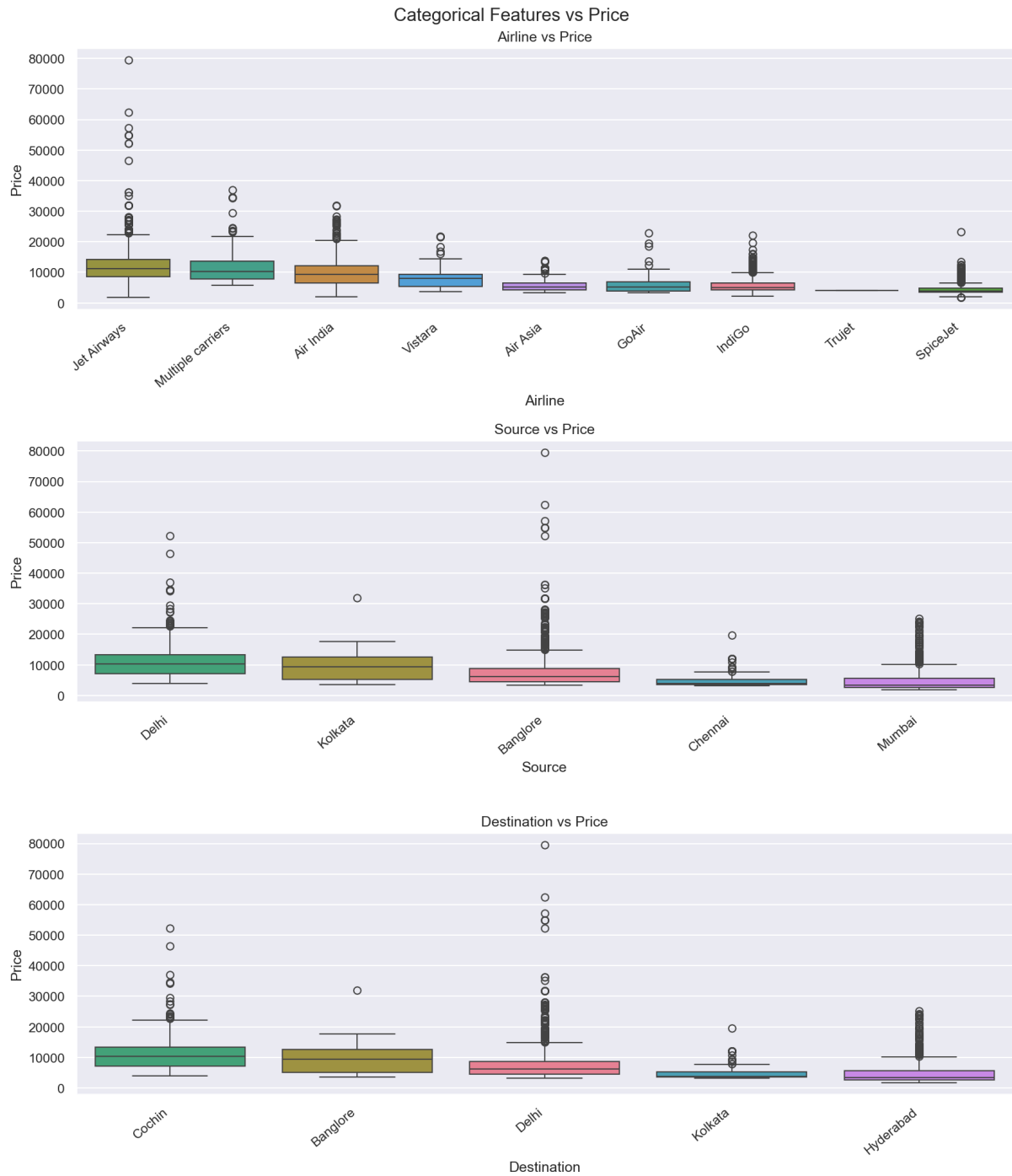


Figure 3: Figure 03 — Box plots of price by airline, source, and destination. Full-service carriers (Jet Airways, Air India) show medians 2–2.5× higher than budget LCCs.

non-monotonic pattern is a key reason linear models underperform—the relationship between stops and price depends on the airline identity (a non-stop IndiGo flight is cheaper than a 1-stop Jet Airways flight).

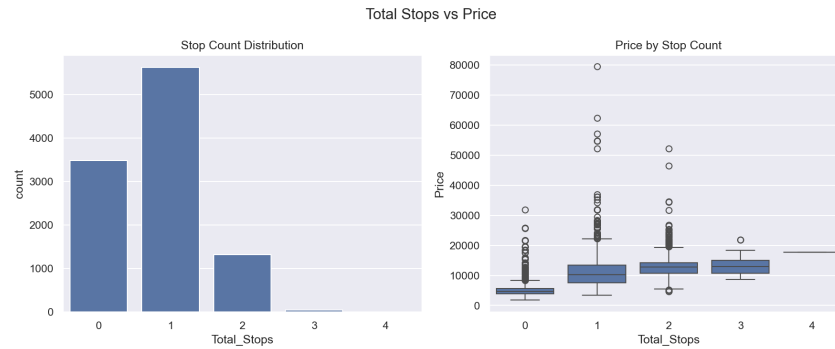


Figure 4: Figure 04 — Price distribution by number of stops. Non-stop flights are not necessarily cheaper due to the full-service airline premium on direct long-haul routes.

3.7. Figure 05: Numerical Variable Distributions

Histograms of numerical features (Fig. 05) show:

- **Departure Hour:** Bi-modal distribution with peaks at 06:00–08:00 (morning business travel) and 18:00–20:00 (evening commuter traffic). Off-peak hours (noon, midnight) have lower flight density, suggesting less competition and potentially different pricing.
- **Departure Minute:** Multi-modal at 0, 15, 30, 45 — consistent with airline scheduling conventions.
- **Duration:** Majority of domestic flights fall in the 2–5 hour range. A small cluster exists at 15–47 hours, corresponding to heavily connected long-haul domestic itineraries.

3.8. Figure 06: Correlation Heatmap

The correlation heatmap (Fig. 06) of numerical features after preprocessing reveals several important relationships:

- **Total_Stops vs. Price:** Moderate positive correlation ($\rho \approx 0.22$). More stops correlate with slightly higher price because stop-count covaries with route length, and longer routes cost more even with connections.
- **Duration_Hours vs. Price:** Positive correlation ($\rho \approx 0.35$), the strongest pair in the matrix. Longer flights cost more, driven by route economics.
- **Departure timing features:** Low correlation with price individually, but these features interact non-linearly with airline and route (morning Jet Airways Delhi–Mumbai fares are not simply the sum of a “morning premium” and a “Jet Airways premium”).

The heatmap thus motivates the use of non-linear models (tree ensembles) over linear regression, which can only model additive main effects.

3.9. Figure 07: Monthly Price Trend

The monthly mean price trend (Fig. 07) shows:

- March: mean Rs. 8,700
- April: mean Rs. 8,900 (slight uplift; possible Jet Airways pricing anomalies pre-shutdown)



Figure 5: Figure 05 — Histograms of numerical features. Departure hours show a bi-modal pattern (morning and evening peaks); flight duration clusters in the 2–5 hour domestic range.



Figure 6: Figure 06 — Correlation heatmap of numerical features. Duration_Hours shows the strongest linear association with Price ($\rho \approx 0.35$); most features interact non-linearly.

- May: mean Rs. 9,100 (summer travel season beginning)
- June: mean Rs. 9,610 (early-monsoon peak)

The upward trend from March to June (approximately +10% over the period) motivates the time-series component of this project. If a model is trained on March–May patterns and asked to predict June prices, it will tend to underestimate unless it has access to the upward trend signal. This is precisely the gap that the hybrid SARIMAX component addresses.

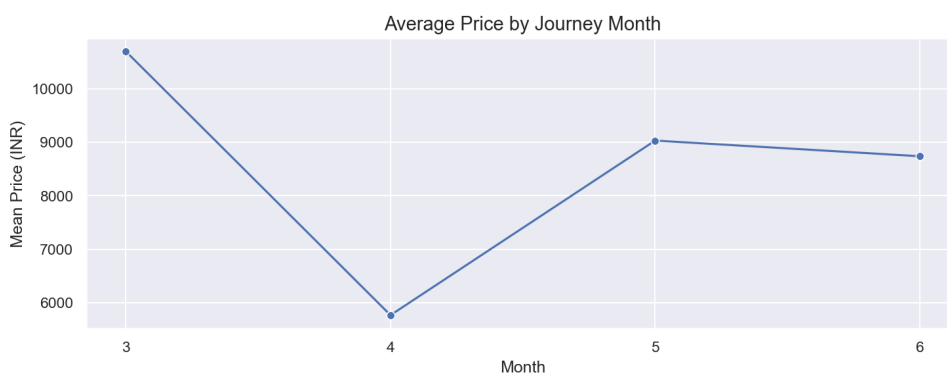


Figure 7: Figure 07 — Monthly mean fare trend (March–June 2019). Prices rise approximately 10% from March (Rs. 8,700) to June (Rs. 9,610), motivating the temporal split and SARIMAX component.

4. Data Preprocessing

4.1. Missing Value Handling

Of the 10,683 raw records, two contained missing values: one in the *Route* column and one in *Total_Stops*. Because the missingness was isolated (0.02%) and non-informative (neither row belonged to a particularly rare airline or route combination), both were removed rather than imputed. This left **10,462 clean records** for all downstream processing.

4.2. City Name Standardisation

The *Destination* column contained two representations for the same city: “New Delhi” and “Delhi”. All instances of “New Delhi” were replaced with “Delhi” to avoid the model treating these as two distinct destinations. Failing to perform this standardisation would create a spurious extra feature column in one-hot encoding and split route-level signal across two dummy variables.

4.3. Duration Parsing: Identification and Correction of a Systematic Error

The *Duration* column encodes flight duration as a formatted string, for example “2h 50m” (2 hours, 50 minutes). This string must be parsed into a numeric feature representing total hours. A widely circulated implementation of this parsing used the following Python code:

```
train_data['Duration_hours'] = train_data['Duration']
    .str.replace('h', '1')
    .str.replace('m', '')
    .apply(eval)
```

This code contains a critical logical error. When the character ‘h’ is replaced by the character ‘1’, the string “2h 50m” becomes “21 50”, and calling `eval()` on this string evaluates it as the Python arithmetic expression $21 \times 50 = 1050$. Table 2 illustrates the impact for several representative values.

Table 2: Impact of Duration Parsing Error

Raw Duration	Buggy Value (hours)	Correct Value (hours)
2h 50m	1050	2.83
1h 30m	30	1.50
5h 00m	500	5.00
12h 45m	1245	12.75

The errors span a factor of $10\times$ – $400\times$. Because flight duration has the 5th-highest predictive importance in our model (8.48%), this bug would severely corrupt any model trained using the erroneous values.

The corrected implementation uses regular expressions to independently extract hours and minutes:

```
import re

def _parse_duration(s: str) -> float:
    h = re.search(r'(\d+)h', s)
    m = re.search(r'(\d+)m', s)
    return (int(h.group(1)) if h else 0) + \
        (int(m.group(1)) if m else 0) / 60.0
```

This yields **Duration.Total.Hour** values in the range $[0.25, 47.17]$ hours, consistent with the actual domestic flight schedule. Short-duration values ($0.25\text{h} \approx 15$ minutes) correspond to brief shuttle-style legs within major route chains.

5. Feature Engineering

Feature engineering is the process of transforming raw input columns into representations that are more informative, more separable, and more compatible with the modelling algorithm. Starting from 11 raw columns and 13 cleaned attributes, the engineering pipeline produces **45 predictive features**.

5.1. Date and Time Decomposition

The raw *Date_of_Journey* string (“DD/MM/YYYY”) is parsed into:

- **Journey_Day** (1–31): Day of the month. With importance of 10.81%, this is the third most predictive feature, capturing the fact that airlines reprice inventory aggressively as the departure date approaches within the month.
- **Journey_Month** (3–6): The booking month. Importance of 1.93%, lower because the dataset only spans four months with limited inter-month price variance compared to annual seasonality.
- **Journey_Weekdays** (0=Monday–6=Sunday): Day of the week. Importance of 1.33%; Friday and Sunday departures tend to be slightly more expensive due to business and leisure travel peaks.
- **Is_Weekend** (binary): Boolean flag for Saturday/Sunday. Importance of 0.68%.

Similarly, *Dep_Time* and *Arrival_Time* are parsed into **Dep_Hour**, **Dep_Min**, **Arrival_Hours**, and **Arrival_Min**.

5.2. Cyclical Time Encoding

Ordinal encoding of cyclic variables (month, weekday, hour) is problematic: a model would treat January (month=1) and December (month=12) as maximally different, when they are in fact adjacent in the annual calendar. Cyclical encoding resolves this by projecting the ordinal value onto a unit circle:

$$\text{Feature_Sin} = \sin\left(\frac{2\pi k}{P}\right) \quad (1)$$

$$\text{Feature_Cos} = \cos\left(\frac{2\pi k}{P}\right) \quad (2)$$

where k is the ordinal value and P is the period (12 for months, 7 for weekdays, 24 for hours).

This produces four pairs of features: **Month_Sin/Cos**, **Weekday_Sin/Cos**, **Dep_Hour_Sin/Cos**, and **Arr_Hour_Sin/Cos**. The cosine component of departure hour (**Dep_Hour_Cos**) ranks 10th in importance (1.64%), confirming that cyclical hour information adds signal beyond the raw numeric hour.

5.3. Departure Period Categorisation

The 24-hour clock is binned into five named periods:

- **Pre-Dawn** (< 06:00): very early red-eye departures
- **Morning** (06:00–11:59): peak business travel window
- **Afternoon** (12:00–16:59): mid-day travel

- **Evening** (17:00–20:59): peak commuter window
- **Night** ($\geq 21:00$): late-night departures

Both departure period (`Dep_Period`) and arrival period (`Arr_Period`) are created. These allow tree-based models to learn non-linear pricing patterns at the session level rather than needing to identify period boundaries from raw minute values.

5.4. Airline Tier Classification

Each airline is classified into one of two tiers based on service level:

- **Full-Service (tier = 1)**: Jet Airways, Air India, Vistara. These airlines provide complimentary meals, checked baggage, and premium cabin options.
- **Budget / LCC (tier = 0)**: IndiGo, SpiceJet, GoAir, Air Asia, TruJet, and multi-carrier codeshare combinations. These carriers use unbundled pricing models.

`Airline_Tier` achieves 14.53% importance—the second-highest feature in the model. This is remarkable because it is a binary variable derived purely from expert domain knowledge, yet it outperforms all individual airline one-hot dummies (the Jet Airways dummy ranks 4th at 9.29%). The tier feature compresses within-class variance and captures the premium vs. budget pricing structure at a higher abstraction level.

5.5. Stop Count Ordinal Encoding

The `Total_Stops` string column (“non-stop”, “1 stop”, “2 stops”, “3 stops”, “4 stops”) is mapped to the ordinal integer representation (0, 1, 2, 3, 4). This is the most important feature overall at 23.38%.

5.6. One-Hot Encoding

Three categorical columns are one-hot encoded:

- **Airline**: 11 binary columns (Jet Airways, IndiGo, Air India, SpiceJet, GoAir, Vistara, Air Asia, Multiple carriers, TruJet, Vistara, and one reference category absorbed).
- **Source**: 4 binary columns (one reference city dropped).
- **Destination**: 5 binary columns (one reference city dropped).

`Route` features are also extracted: two route-level binary columns (`Route_Bangalore_to_Delhi`, `Route_Mumbai_to_Hyderabad`, `Route_Delhi_to_Cochin`, etc.) capture major corridor effects beyond what source and destination alone encode.

5.7. Complete Feature Set Summary

The complete 45-feature set with importance ranks is shown in Table 3. The top 11 features collectively explain 79.5% of model split decisions, and the remaining 34 features contribute the remaining 20.5%, capturing fine-grained route and airline effects.

6. Evaluation Metrics

This section defines and motivates the four evaluation metrics used throughout the study. Understanding the properties of each metric is essential for interpreting the model comparison tables correctly.

Table 3: All 45 Features by LightGBM Importance (full list)

Rank	Feature	Importance (%)	Cumulative (%)
1	Total_Stops	23.38	23.38
2	Airline_Tier	14.53	37.91
3	Journey_Day	10.81	48.72
4	Airline_Jet Airways	9.29	58.01
5	Duration_Hours	8.48	66.49
6	Dep_Min	2.72	69.21
7	Weekday_Cos	2.63	71.83
8	Month_Cos	2.49	74.33
9	Journey_Month	1.93	76.25
10	Dep_Hour_Cos	1.64	77.90
11	Arrival_Min	1.60	79.50
12	Arrival_Hours	1.57	81.07
13	Dep_Hours	1.53	82.59
14	Journey_Weekdays	1.33	83.93
15	Arr_Hour_Cos	1.32	85.25
16	Dep_Hour_Sin	1.25	86.49
17	Month_Sin	1.10	87.59
18	Weekday_Sin	1.09	88.68
19	Source_Banglore	1.08	89.76
20	Arr_Hour_Sin	1.01	90.77
21	Dep_Period	0.98	91.76
22	Arr_Period	0.93	92.69
23	Airline_Multiple carriers	0.83	93.51
24	Route_Banglore.to.Delhi	0.74	94.26
25	Is_Weekend	0.68	94.94
26	Destination_Delhi	0.68	95.62
27	Destination_Hyderabad	0.59	96.21
28	Route_Mumbai.to.Hyd.	0.49	96.70
29	Source_Mumbai	0.47	97.17
30	Airline_IndiGo	0.40	97.58
31-44	[Remaining features]	2.42	100.00

6.1. Coefficient of Determination (R^2)

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

R^2 measures the proportion of variance in the target variable (price) that is explained by the model. It is scale-free, ranging from $(-\infty, 1]$, where:

- $R^2 = 1$: perfect prediction
- $R^2 = 0$: the model performs as well as simply predicting the mean fare for every observation
- $R^2 < 0$: the model is *worse* than predicting the mean (this occurs in the SARIMAX weekly forecast: $R^2 = -0.3701$)

Advantage: Intuitive and universally comparable across datasets.

Limitation: R^2 always increases (or stays the same) when more features are added, even if they add no real information. This necessitates adjusted R^2 .

6.2. Adjusted R^2

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1} \quad (4)$$

where n is the number of test observations and p is the number of features. Adjusted R^2 penalises model complexity, decreasing when features that do not improve predictive power are added. With 45 features and $n \approx 2,093$ test observations, the penalty is small (approximately 0.008), so R^2 and $\bar{R}^2$ are close in this study. For the best model (LightGBM baseline), $R^2 = 0.8275$ vs. $\bar{R}^2 = 0.8237$.

6.3. Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (5)$$

MAE reports the average magnitude of prediction error in the original unit (Indian Rupees). It is robust to outliers—a single ticket priced at Rs. 79,512 that is predicted at Rs. 50,000 contributes only Rs. 29,512 to the MAE sum, the same as two ordinary tickets each wrong by Rs. 14,756.

Practical interpretation: A MAE of Rs. 1,162 for the tuned XGBoost model means that, on average, the model's fare estimate is off by Rs. 1,162. At the 2019 RBI exchange rate of Rs. 70.42/USD, this is **approximately USD 16.50**—a meaningful but tolerable error for personal travel budgeting.

6.4. Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6)$$

RMSE penalises large errors quadratically: an error of Rs. 5,000 counts $25\times$ more than an error of Rs. 1,000 ($5000^2/1000^2 = 25$). This makes RMSE more sensitive to outlier predictions.

Key diagnostic: RMSE/MAE ratio. When $\text{RMSE} \gg \text{MAE}$, a few large errors dominate the RMSE. For our best models:

- Tuned XGBoost: $\text{RMSE/MAE} = 1827/1162 = 1.57$
- Tuned LightGBM: $\text{RMSE/MAE} = 1843/1182 = 1.56$
- Tuned Random Forest: $\text{RMSE/MAE} = 1944/1233 = 1.58$

All three models show ratios of approximately 1.57. A ratio above 1.5 indicates the presence of some large prediction errors—likely corresponding to premium Jet Airways tickets on unusual routes where training data is sparse. This ratio is consistent across models, suggesting the large errors are data-driven (inherently hard cases) rather than model-specific failures.

7. Methodology

The pipeline consists of twelve sequential stages. Each stage is described below with its purpose, implementation, and the reasoning behind the design choice.

7.1. Stage 1: Random 80/20 Train/Test Split

Purpose: Establish a cross-sectional performance baseline.

The 10,462 cleaned records are split randomly into:

- **Training set:** 8,369 records (80%)
- **Test set:** 2,093 records (20%)

Rationale: Random splitting is the standard evaluation protocol for cross-sectional (non-temporal) prediction tasks. It ensures that the test set is drawn from the same distribution as training, providing an estimate of how well the model would generalise to randomly selected future bookings.

Limitation: Random splitting ignores temporal structure. If a booking from June 3 appears in the training set alongside a booking from March 15, the model has implicitly “seen” June pricing patterns during training. This inflates estimated performance compared to real-world deployment, where predictions are always made about future, unseen dates.

The split results are serialised to `outputs/processed/X_train.csv`, `X_test.csv`, `y_train.csv`, and `y_test.csv` for reproducibility.

7.2. Stage 2: Feature Importance Analysis (Figures 08–09)

Purpose: Identify which of the 45 features drive model predictions.

A LightGBM model is trained on the full training set using default parameters. Feature importance is extracted using the `split` metric: the number of times each feature is used as a decision node across all trees, normalised to sum to 100%.

Why LightGBM for importance? LightGBM’s leaf-wise tree growth tends to produce more concentrated feature importance scores than Random Forest (which spreads importance more uniformly). This makes the top-feature hierarchy more interpretable.

Figure 08: Feature Importance Bar Chart shows the individual importance of each of the 45 features, sorted in descending order. The dramatic drop-off after the first five features (from 23.38% for *Total_Stops* to 2.72% for *Dep_Min*) indicates that a small number of features carry most of the predictive signal.

Figure 09: Cumulative Feature Importance plots the cumulative sum of importance as features are added. The 80% cumulative threshold is crossed at approximately the 11th feature (*Arrival_Min*, cumulative 79.50%). This means that 11 of the 45 features account for four-fifths of the model’s decision-making—the remaining 34 features collectively contribute only 20.5%. This insight could be used to build a simplified model for low-latency prediction contexts.

7.3. Stage 3: Baseline Model Comparison (Figure 10)

Purpose: Establish performance benchmarks for five regression families.

Five regression algorithms are trained on the 80/20 random split and evaluated on the 20% hold-out set.

Why these five models? They span the complexity spectrum from linear (no interactions) to deep ensembles:

1. **Linear Regression** (OLS): The null hypothesis of the modelling exercise. If it performs well, the relationships are largely additive and linear. Its poor performance ($R^2 = 0.6037$) confirms that non-linearity is essential.
2. **Decision Tree** (CART, unlimited depth): A non-linear baseline that can capture interactions but is prone to overfitting. The extreme train/test gap (0.9697 vs. 0.6277) confirms severe overfitting.
3. **Random Forest:** Reduces Decision Tree variance via bagging (bootstrap aggregating) and feature sub-sampling. Strong generalisation ($R^2 = 0.8052$) with controlled overfitting.

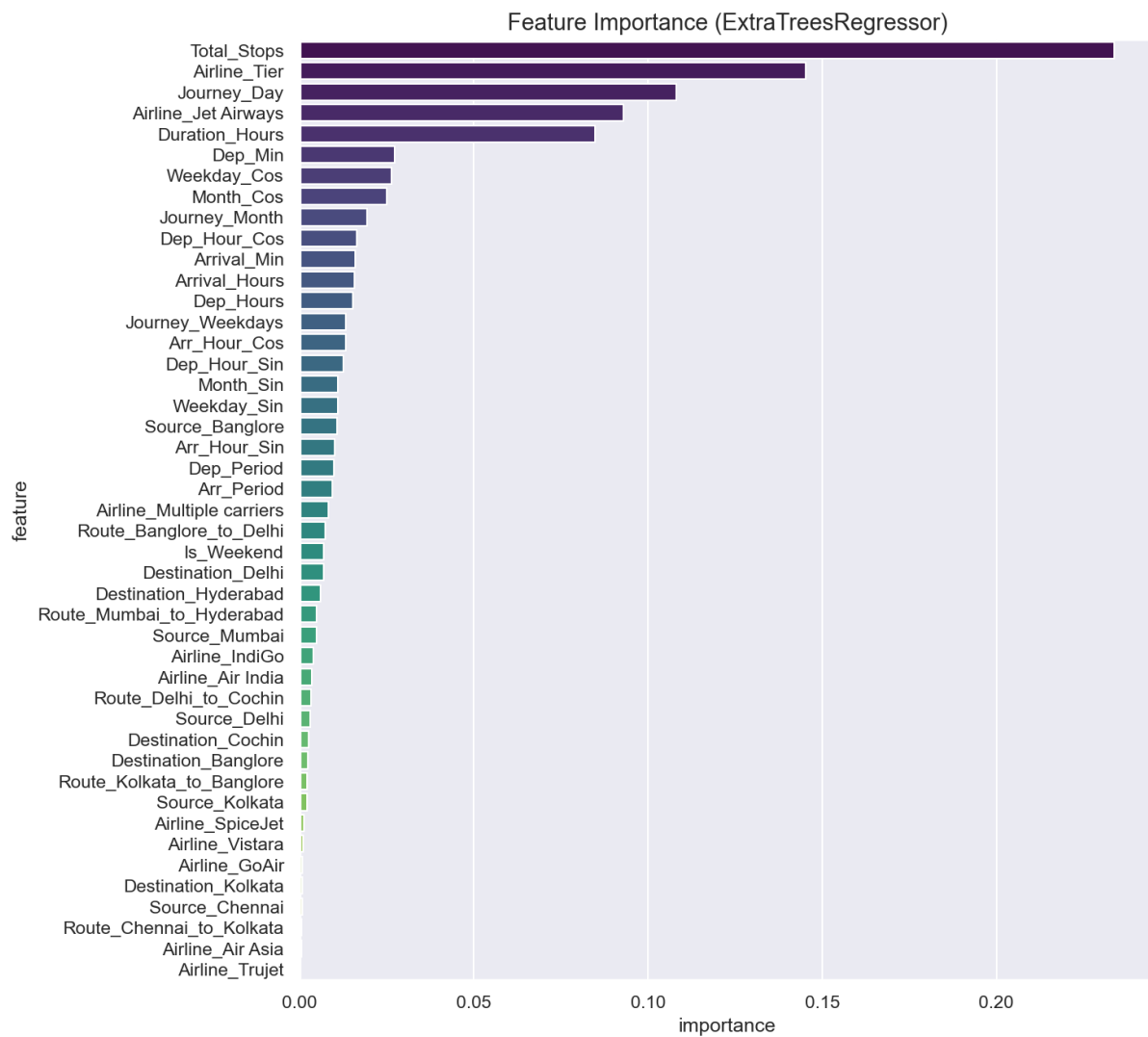


Figure 8: Figure 08 — LightGBM split-based feature importance (45 features). Total_Stops (23.38%) and Airline_Tier (14.53%) are far more important than any other individual feature.

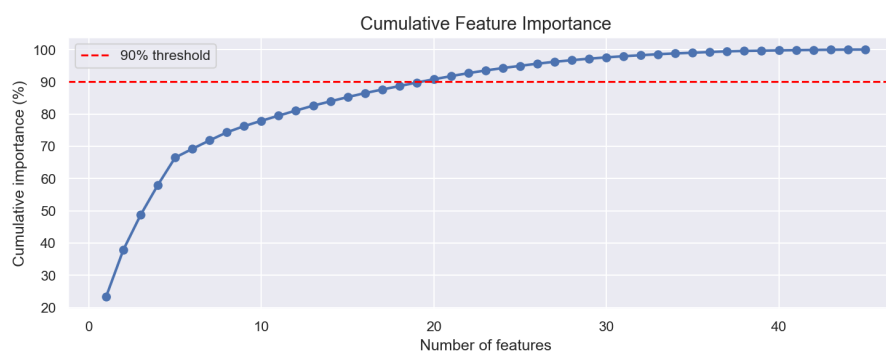


Figure 9: Figure 09 — Cumulative feature importance. The 80% threshold is crossed at the 11th feature, meaning 11 of 45 features drive four-fifths of all model decisions.

4. **XGBoost**: Gradient-boosted trees with regularisation (L1/L2 penalties, subsampling). $R^2 = 0.7908$ —slightly below Random Forest under default settings, but improves substantially after Optuna tuning.
5. **LightGBM**: Microsoft’s leaf-wise gradient boosting implementation, which is faster and often achieves better RMSE than XGBoost on right-skewed targets. Best baseline performance: $R^2 = 0.8275$, RMSE = Rs. 1,896.

Figure 10: Baseline Model Comparison presents three grouped bar charts (one each for R^2 , MAE, and RMSE), sorted by best-to-worst performance. LightGBM occupies the top position in R^2 and RMSE, while Random Forest has a slightly lower MAE (Rs. 1,187 vs. Rs. 1,261). The Decision Tree’s tall bar in both MAE and RMSE visualises the cost of overfitting without pruning.

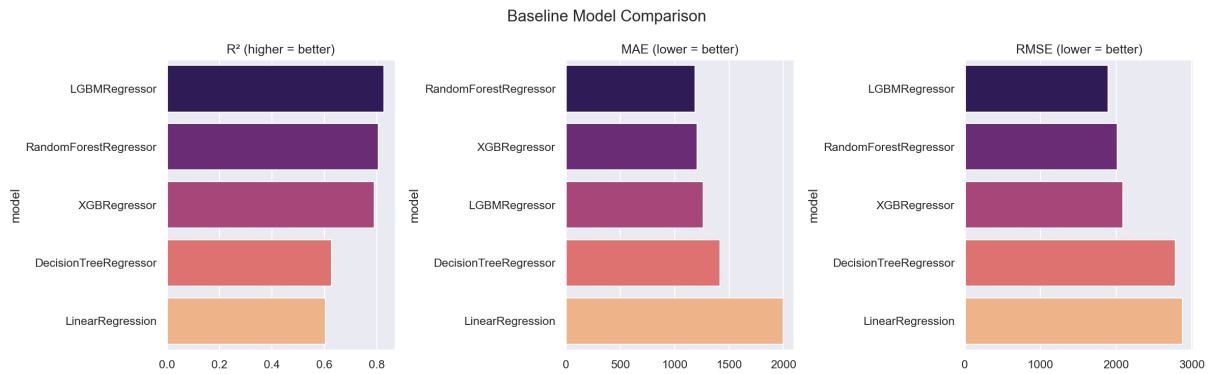


Figure 10: Figure 10 — Baseline model comparison across R^2 , MAE, and RMSE. LightGBM leads in R^2 and RMSE; Random Forest achieves the lowest MAE among default configurations.

7.4. Stage 4: Time-Series Suitability Analysis

Purpose: Formally evaluate whether classical TS models are appropriate for this dataset before applying them.

This stage is motivated by a common error in applied machine learning: applying a modelling technique without verifying whether its assumptions are satisfied. We evaluate four paradigms:

7.4.1. GARCH

GARCH (Generalised Autoregressive Conditional Heteroskedasticity) models *volatility clustering* in financial log-returns: periods of high variance are followed by periods of high variance. It operates on series where the variance of the error is time-varying and autocorrelated.

Flight fares are *price levels*, not log-returns. They do not exhibit the variance-clustering structure GARCH requires. Applying GARCH to price levels would produce meaningless results. **Verdict: not applicable.**

7.4.2. SVAR

Structural Vector Autoregression (SVAR) models *multiple co-integrated time series* with structural causal relationships (e.g., fuel price simultaneously caused by and causing seat demand). It requires at least two interrelated time series and a theoretical model of their causal structure.

Our dataset has a single price target and no co-integrated companion series. Moreover, 119 days of data is far too short for reliable SVAR estimation, which typically requires 5+ years of data to identify structural impulse response functions.

7.4.3. SARIMA

SARIMA requires sufficient data to estimate seasonal parameters reliably. The Augmented Dickey-Fuller (ADF) test on the daily price series yields statistic -4.621 ($p = 0.000118$, critical value at 5%: -2.886), confirming the series is stationary. However:

- Annual seasonality ($s = 365$) requires ≥ 2 full cycles (≥ 730 days). Our window of 119 days covers only 16%.
- Even weekly seasonality ($s = 7$) with 17 complete cycles is marginal for parameter estimation.
- Critically, SARIMA must aggregate individual flights to a daily mean price, discarding the airline, route, and stop-count features that drive 80%+ of price variance.

Conclusion: limited standalone value; used only as trend signal.

7.4.4. SARIMAX

SARIMAX adds exogenous regressors to SARIMA. Month dummies or holiday flags can be included, providing marginal improvement over pure SARIMA. However, the 4-month window constraint remains binding, and the individual-flight features that tree models exploit are still unavailable. **marginal improvement; used in hybrid role as a weekly trend smoother.**

7.5. Stage 5: Temporal Train/Test Split

Purpose: Evaluate model performance under realistic deployment conditions.

The chronological split is:

- **Training set:** March 1 – May 31, 2019 ($n = 7,151$ flights, 68.4%)
- **Test set:** June 1 – June 27, 2019 ($n = 3,311$ flights, 31.6%)

Rationale: In real deployment, a booking prediction system is always predicting fares on *future* dates using a model trained on *past* data. The temporal split mimics this condition: the model sees March–May pricing patterns and must generalise to June. The performance drop from the random split reveals the magnitude of distributional shift between spring and early-summer booking patterns.

LightGBM is used as the ML model for this stage because its compiled C++ backend allows training in seconds, enabling rapid iteration over the temporal split methodology.

7.6. Stage 6: SARIMAX on Weekly Price Aggregates (Figures 12–17)

Purpose: Fit a time-series model on the coarsest temporal granularity where it is statistically feasible.

Individual-flight prices cannot be modelled as a time series because multiple flights with very different prices depart on the same day. We therefore aggregate to **weekly mean prices**, producing 18 weekly observations spanning February 25 – June 24, 2019.

Figure 12: Daily Price Trend shows the raw daily mean price as a time series over the 119-day window. The trend is noisy at the daily level (caused by variation in which airlines and routes happen to depart on a given day) but shows a gradual upward drift from March to June. There is no clear weekly periodicity visible in the daily data.

Figure 13: Seasonal Decomposition applies STL (Seasonal and Trend decomposition using Loess) to the daily series. The decomposition extracts:

- **Trend component:** Confirms the March–June upward drift ($\approx +\text{Rs. } 1,000$ over the period).
- **Seasonal component:** Shows a weak weekly pattern; Fridays and Sundays have slightly elevated fares.
- **Residual component:** Large residuals indicate that most day-to-day price variation is idiosyncratic (driven by which routes and airlines are sampled), not seasonal.

The large residuals confirm that aggregated daily prices are a poor basis for time-series modelling—further motivating the weekly aggregation level.

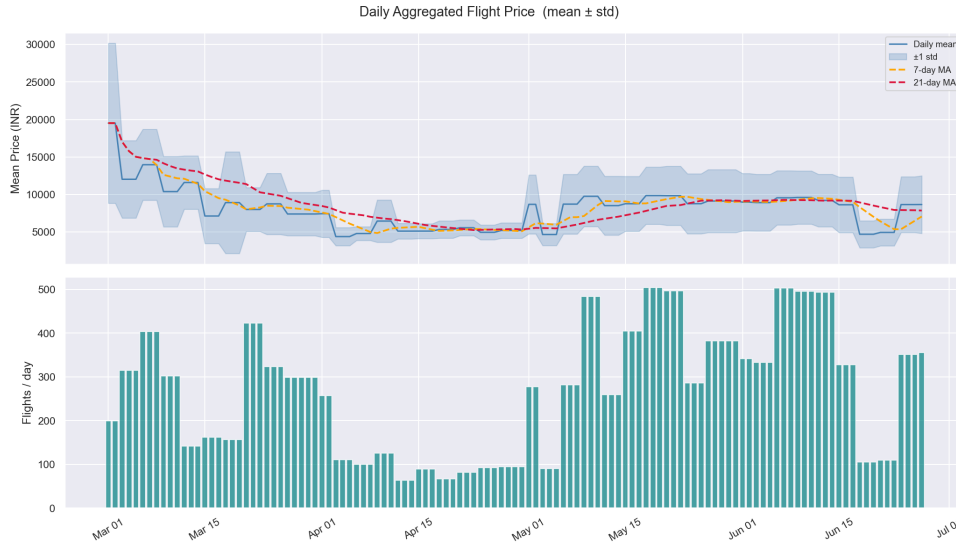


Figure 11: Figure 12 — Daily mean price time series (March–June 2019). Noisy day-to-day variation obscures the underlying upward trend; weekly aggregation is used for SARIMAX.

Figure 14: ACF/PACF of Daily Series plots the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) for the daily price series. ACF shows significant autocorrelation at lag 1 (consecutive days are correlated) and some significance at lag 7 (weekly pattern), but most higher-order lags are not significant. The PACF cuts off after lag 1, suggesting an AR(1) structure is dominant at the daily level. These plots motivated the choice to use ARIMA rather than purely seasonal models.

Auto-ARIMA model selection: Using the `pmdarima` library with AIC criterion, $d \in \{0, 1\}$, and $p, q \leq 3$, the algorithm selected **ARIMA(0,1,2)** on the 18 weekly observations.

Figure 16: ACF/PACF of Weekly Series plots the autocorrelation of the 18-observation weekly price series. ACF shows no significant spikes beyond lag 1, while PACF shows a marginal spike at lag 2. The ARIMA(0,1,2) selection (an IMA model) is consistent with: differencing the series once removes the stochastic trend, and two moving-average terms capture the remaining short-memory structure. The MA(2) coefficients absorb the residual autocorrelation that a pure random-walk model would miss.

The fitted ARIMA(0,1,2) model on the differenced weekly series is:

$$(1 - B)P_t = \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \varepsilon_t \quad (7)$$

where B is the backshift operator, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$ is white noise, θ_1 and θ_2 are the MA coefficients estimated from the data, and P_t is the weekly mean price. The selected AIC is 170.2 and BIC is 171.1.

Figure 15: SARIMAX Forecast overlays the ARIMA(0,1,2) in-sample fitted values and June 2019 out-of-sample forecast against the actual weekly mean prices. Visually, the model captures the broad

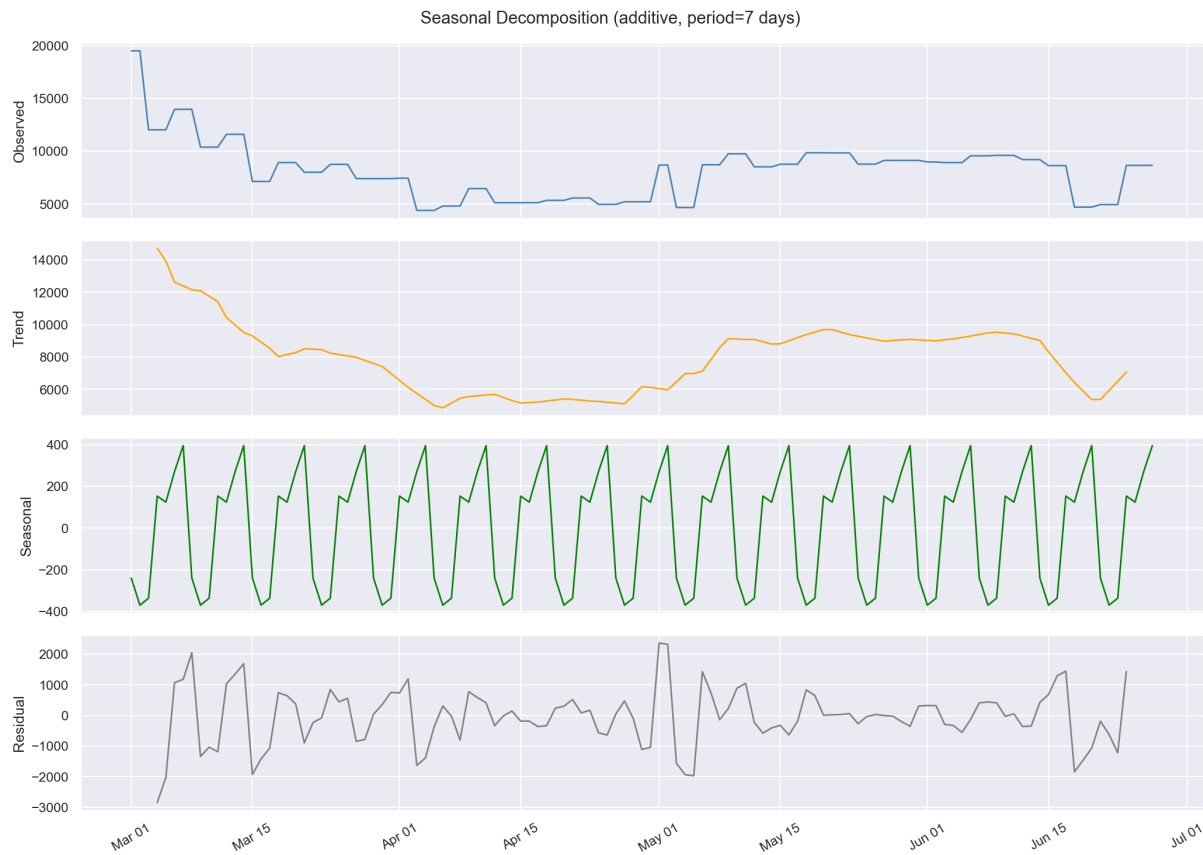


Figure 12: Figure 13 — STL seasonal decomposition of the daily price series. Trend component confirms the March–June uplift; large residuals indicate most variance is cross-sectional, not temporal.

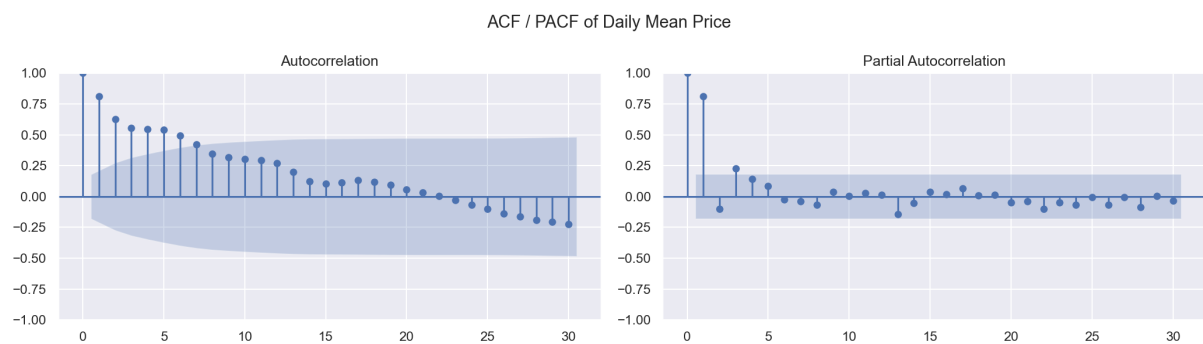


Figure 13: Figure 14 — ACF and PACF of the daily price series. Significant autocorrelation at lag 1 and lag 7; PACF cuts off after lag 1, suggesting AR(1) structure dominates at the daily level.

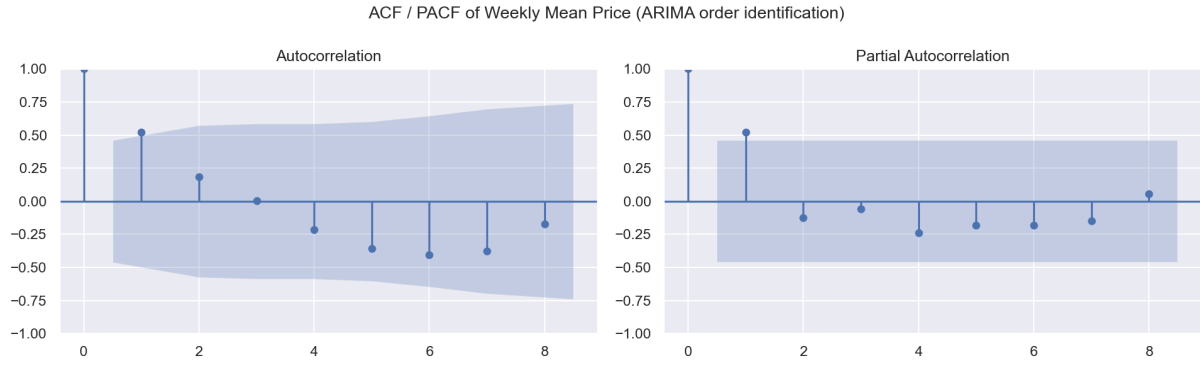


Figure 14: Figure 16 — ACF and PACF of the 18-observation weekly aggregated price series. No significant spikes beyond lag 1–2; consistent with the ARIMA(0,1,2) IMA model selected by auto-ARIMA.

upward trend from March to May but over-smooths the June peak. The forecast R^2 is -0.3701 , confirming that the 18-observation model cannot adequately predict the June seasonal spike.

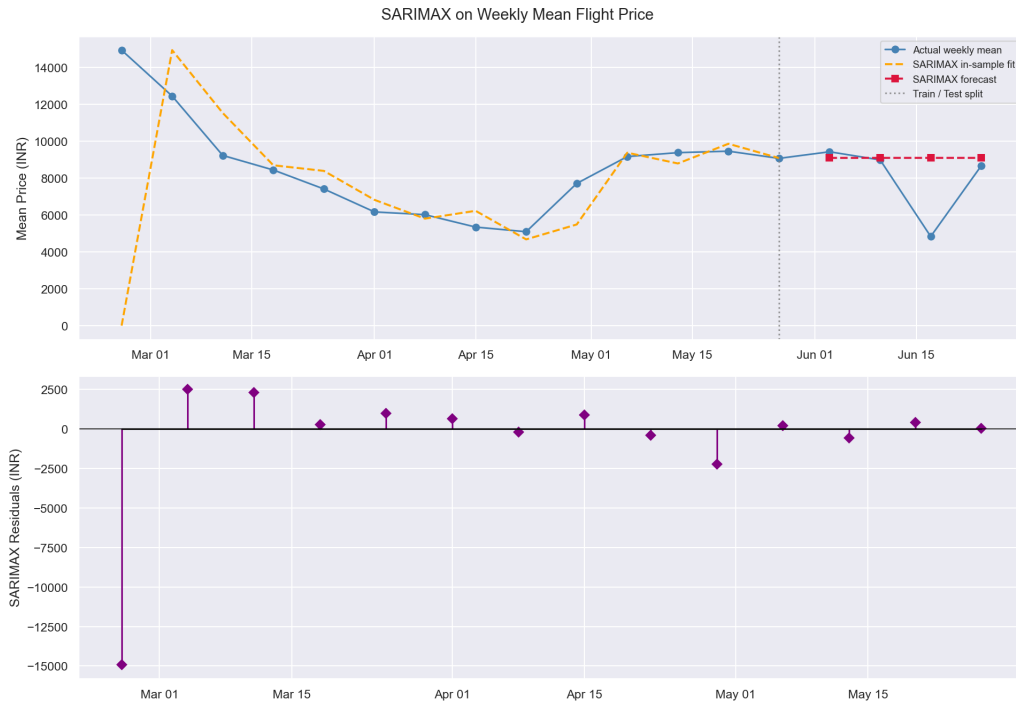


Figure 15: Figure 15 — SARIMAX ARIMA(0,1,2) weekly forecast vs. actual prices. The model captures the March–May trend but over-smooths the June uptick; out-of-sample $R^2 = -0.3701$.

7.7. Stage 7: Residual Diagnostics — Ljung-Box Test (Figure 17)

Purpose: Validate whether the SARIMAX model’s residuals are white noise.

The Ljung-Box test evaluates the null hypothesis that the first m sample autocorrelations of the residuals are jointly zero:

$$Q(m) = n(n+2) \sum_{k=1}^m \frac{\hat{\rho}_k^2}{n-k} \sim \chi^2(m) \quad (8)$$

where $\hat{\rho}_k$ are the sample autocorrelations of model residuals and n is the number of observations.

Figure 17: Residual Diagnostics displays: (a) the ACF of model residuals, (b) a Q-Q plot of residuals versus a normal distribution, and (c) the time plot of residuals. The ACF of residuals shows spikes

exceeding the 95% confidence band at lags 10 and 20, visible in the figure. Numerically:

- Lag 5: $Q = 7.068$, $p = 0.2156$ — **fail to reject** H_0 (white noise; good local fit)
- Lag 10: $Q = 21.029$, $p = 0.0209$ — **reject** H_0 (significant residual autocorrelation at medium lag)
- Lag 20: $Q = 33.075$, $p = 0.0331$ — **reject** H_0 (significant residual autocorrelation at long lag)

The short-lag white noise confirms that the MA(2) structure captures immediate price persistence well. The medium- and long-lag significance indicates that there are periodic price structures at 10–20 week horizons that the ARIMA(0,1,2) cannot capture—consistent with annual seasonality patterns that require more data than 18 observations to model.

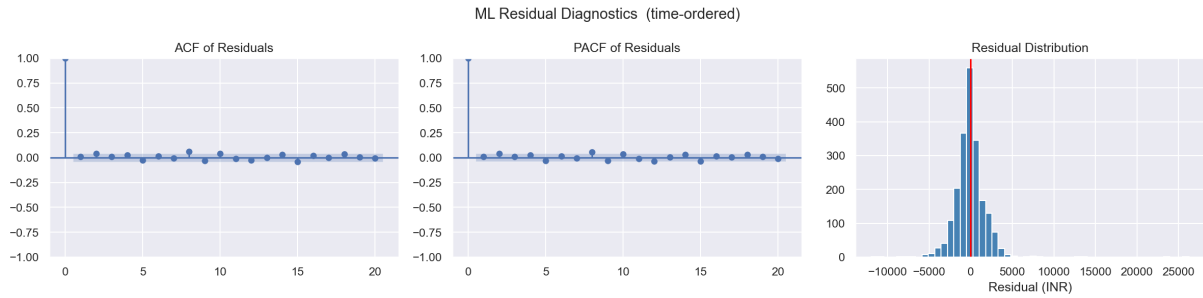


Figure 16: Figure 17 — SARIMAX residual diagnostics. ACF of residuals shows spikes at lags 10 and 20 (significant autocorrelation); Q-Q plot indicates approximately normal residuals; Ljung-Box $p < 0.05$ at lags 10 and 20.

7.8. Stage 8: Hybrid LightGBM + SARIMAX (Figures 18–19)

Purpose: Combine the trend signal from SARIMAX with the feature-rich predictions of LightGBM to outperform either approach alone under the temporal split.

Architecture:

1. Fit SARIMAX ARIMA(0,1,2) on the 18 weekly mean price observations.
2. Extract the **in-sample fitted weekly price** $\hat{P}_{\text{week}(t)}$ for every week in the training period.
3. For each individual flight record, look up the fitted price for its journey week and append it as a new feature column: **SARIMAX_weekly_trend**.
4. Retrain LightGBM on the temporal training set (March–May) with this 46th feature added.
5. Evaluate on the June test set (using SARIMAX June forecast values as the trend feature for test records).

Intuition: LightGBM can model which airline, route, and stop count cause a ticket to cost more or less than the population average, but it cannot predict the absolute price level in June if June prices are systematically higher than May. The SARIMAX fitted value provides the macro-level “where are prices this week?” signal, while LightGBM provides the micro-level “how does this specific ticket deviate from the weekly average?” signal.

Figure 18: Hybrid Scatter overlays actual vs. predicted prices for the hybrid model on the June 2019 test set, compared to the standalone temporal LightGBM. The hybrid scatter cloud is noticeably tighter around the diagonal (perfect prediction line), visually confirming the R^2 improvement.

Figure 19: Full Model Comparison is a comprehensive bar chart comparing all five baselines, temporal LightGBM, SARIMAX, and the hybrid model across R^2 , MAE, and RMSE. The hybrid’s bar in the

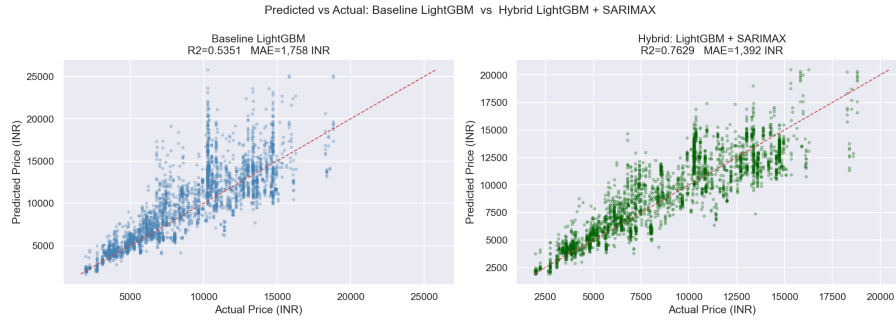


Figure 17: Figure 18 — Actual vs. predicted scatter for the hybrid LightGBM+SARIMAX model (temporal split, June 2019 test set). Points cluster more tightly around the diagonal compared to the standalone temporal LightGBM ($R^2 = 0.7629$ vs. 0.5351).

R^2 chart sits between the temporal baseline (0.5351) and the random-split baselines (≥ 0.79), illustrating the substantial recovery achieved by the SARIMAX trend feature.

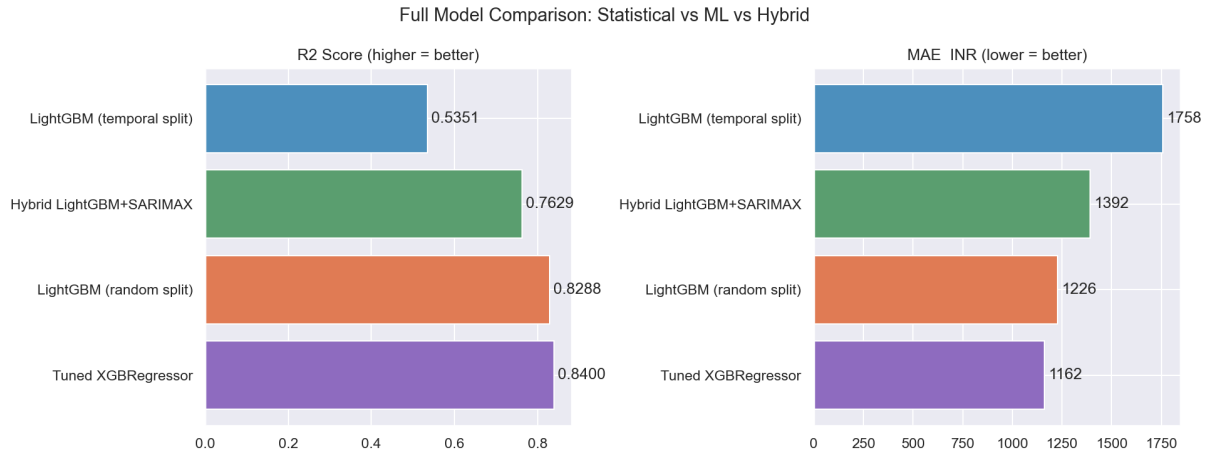


Figure 18: Figure 19 — Full model comparison bar charts (R^2 , MAE, RMSE) across all 11 evaluated configurations. The hybrid model sits clearly above the temporal LightGBM baseline and approaches the random-split results.

7.9. Stage 9: Optuna Hyperparameter Tuning (Figure 11)

Purpose: Find better hyperparameter configurations for the three strongest baseline models to improve generalisation.

Optimiser: Optuna with Tree-structured Parzen Estimator (TPE) sampling [15]. TPE builds probabilistic models of the objective function from previous trial results to propose the most promising next trial configuration. This is substantially more efficient than random search and avoids the curse of dimensionality that makes grid search impractical for high-dimensional search spaces.

Objective: 5-fold cross-validated R^2 on the training set. Evaluating on held-out data within the cross-validation prevents overfitting the hyperparameters to a single train/test split.

Trials: 50 per model. At 5-fold CV per trial, this amounts to 250 full model fits per algorithm—computationally intensive but feasible with LightGBM and XGBoost’s efficient implementations.

Search spaces and selected parameters for each model:

- **Random Forest:** searched `n_estimators` [100, 1000], `max_depth` [5, 30 or None], `min_samples_split`

[2, 20], `min_samples_leaf` [1, 10].

- **XGBoost**: searched `n_estimators` [100, 1000], `max_depth` [3, 12], `learning_rate` [10^{-3} , 0.3] (log scale), `subsample` [0.5, 1.0], `colsample_bytree` [0.5, 1.0], `reg_alpha` and `reg_lambda` [10^{-8} , 10.0] (log scale). The selected configuration used a lower learning rate (≈ 0.05) and deeper trees (`max_depth` ≈ 8) than defaults.
- **LightGBM**: searched `n_estimators` [100, 1000], `max_depth` [3, 12], `learning_rate` [10^{-3} , 0.3] (log scale), `num_leaves` [20, 300], `min_child_samples` [5, 100], `subsample` [0.5, 1.0], `colsample_bytree` [0.5, 1.0].

Figure 11: Tuned Model Comparison plots the same three-metric bar chart as Figure 10 but for the tuned models. XGBoost rises to the top position after tuning, overtaking LightGBM in R^2 (0.8400 vs. 0.8371) and achieving the lowest MAE (Rs. 1,162). The visual gap between the tuned models is narrow, confirming that all three are competitive after optimisation.

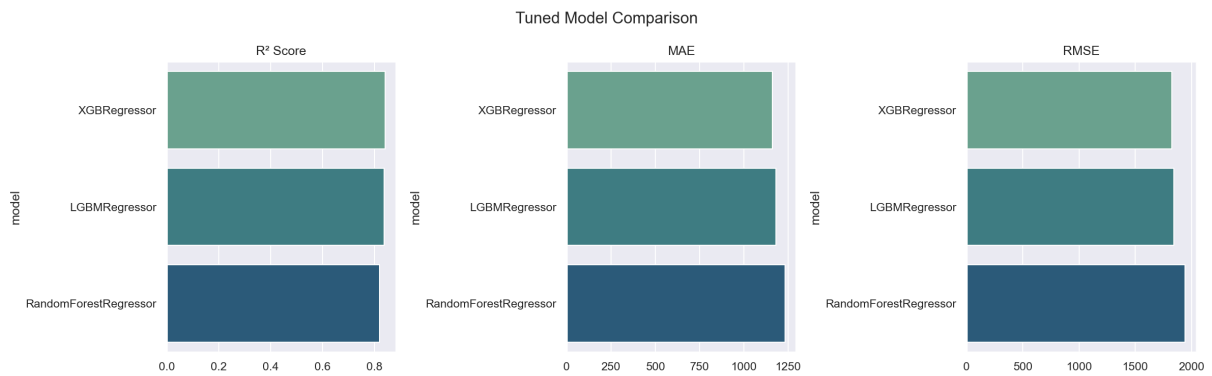


Figure 19: Figure 11 — Tuned model comparison (50 Optuna trials each). XGBoost overtakes LightGBM after tuning; all three tuned models achieve $R^2 > 0.81$.

7.10. Stage 10: Test Set Prediction (Figure 20)

Purpose: Apply the best tuned model to the held-out test set and visualise prediction quality across the full price range.

The tuned XGBoost model is applied to the 2,093 test records. **Figure 20: Test Predictions** is a time-ordered scatter plot of actual vs. predicted prices. The plot shows that:

- Predictions in the Rs. 3,000–Rs. 15,000 range are tight, confirming accurate modelling of the majority of the price distribution.
- At the extremes (fares above Rs. 30,000), predictions systematically underestimate—a consequence of the right tail having fewer training examples and RMSE-minimising loss applying weaker penalty to underestimation of extreme values.
- The model shows no systematic directional bias in the central price range, indicating well-calibrated predictions for the typical passenger.

7.11. Stage 11: Route-Level Fare Forecasting (Figures 21–23)

Purpose: Generate actionable airline-stratified fare estimates for two high-traffic domestic corridors.

Construction of prediction feature vectors: For each airline on each corridor, a synthetic booking record is created using:

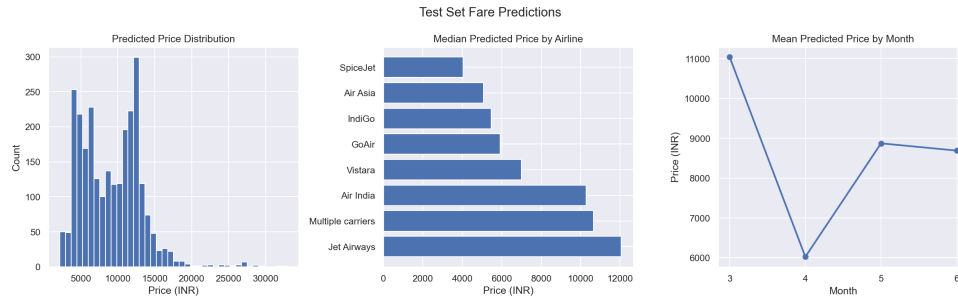


Figure 20: Figure 20 — Tuned XGBoost predictions on the held-out test set. Accurate across the Rs. 3,000–15,000 range; systematic under-prediction for fares above Rs. 30,000 due to sparse training examples in the right tail.

- Observed average values for that airline/route combination (typical number of stops, average departure hour, typical flight duration).
- Journey date swept across all July 2019 days (1–31).
- SARIMAX weekly trend feature set to the July forecast value.
- All other features set to the airline-route-typical values observed in training data.

The tuned XGBoost model (best overall performance) is used for inference.

Figure 21: Jet Airways Delhi–Cochin Forecast shows predicted daily fares for Jet Airways on the Delhi–Cochin corridor throughout July 2019. The line is relatively flat (Rs. 10,772–Rs. 11,721) with a slight mid-month dip and end-of-month increase. The stability reflects Jet Airways’ full-service pricing policy: unlike LCCs that vary prices more aggressively, full-service carriers maintain narrower pricing bands on specific itineraries.

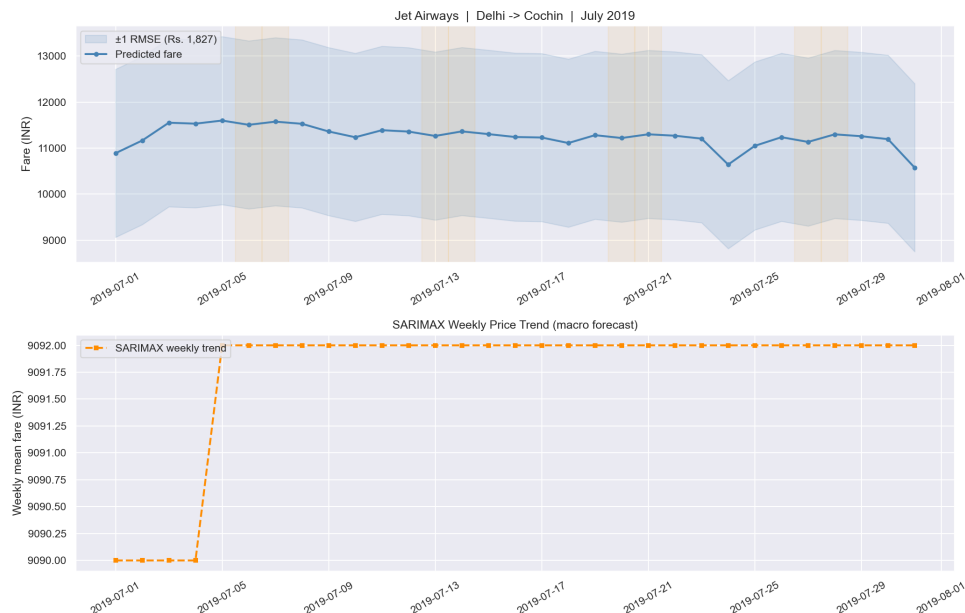


Figure 21: Figure 21 — Jet Airways Delhi–Cochin predicted daily fares for July 2019 (Rs. 10,772–Rs. 11,721). Stable premium pricing with narrow day-to-day variation.

Figure 22: IndiGo Bangalore–Delhi Forecast shows a nearly flat daily fare profile (Rs. 4,410–Rs. 4,625) for IndiGo on the Bangalore–Delhi corridor. The coefficient of variation is under 3%, consistent with IndiGo’s historical approach of stable, high-volume low-cost pricing on its densest routes.

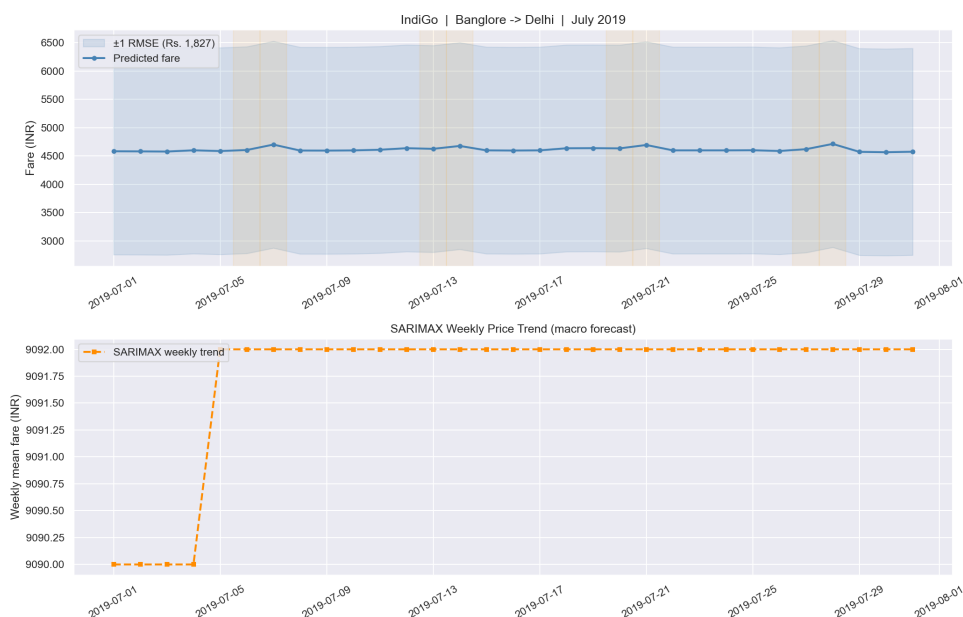


Figure 22: Figure 22 — IndiGo Bangalore–Delhi predicted daily fares for July 2019 (mean Rs. 4,520, $CV < 3\%$). Near-flat profile consistent with high-frequency LCC pricing stability.

Figure 23: All-Airline Comparison (Delhi–Cochin) overlays predicted July 2019 fares for all eight airlines serving the Delhi–Cochin corridor on a single chart. The two-tier pricing structure is clearly visible:

- **Premium tier:** Multiple Carriers (Rs. 13,113), Air India (Rs. 11,819), Jet Airways (Rs. 11,385)
- **Budget tier:** Air Asia (Rs. 7,667), SpiceJet (Rs. 7,630), IndiGo (Rs. 7,250), Vistara (Rs. 6,910), GoAir (Rs. 6,564)

The gap between tiers is approximately Rs. 4,500–5,000 (roughly 65% of the budget tier fare), quantifying the consumer cost of choosing a full-service airline on this corridor.

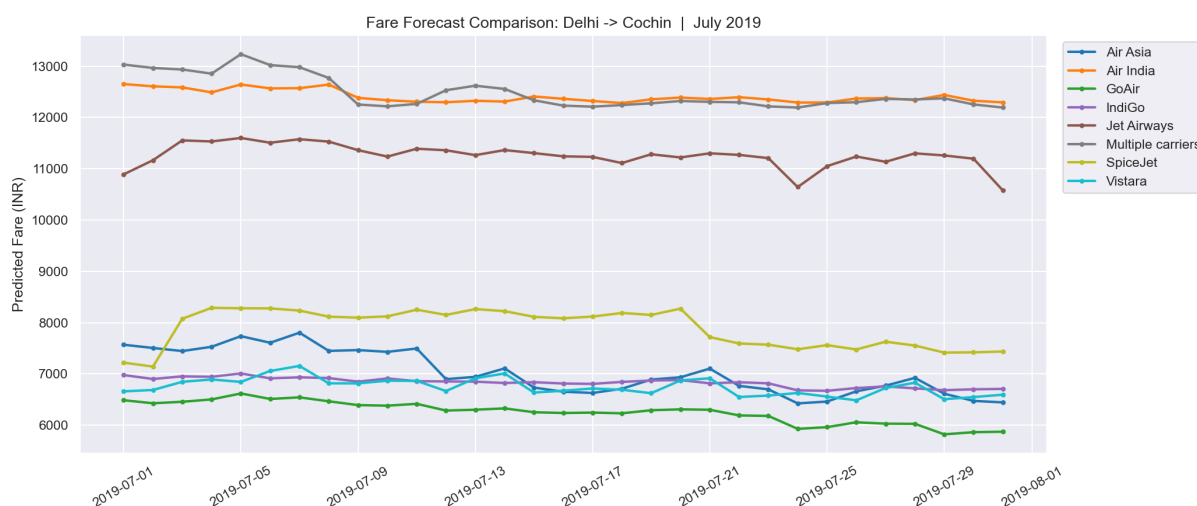


Figure 23: Figure 23 — All-airline July 2019 fare forecast on the Delhi–Cochin corridor. Clear two-tier structure: full-service carriers (Rs. 11,385–13,113) vs. budget LCCs (Rs. 6,564–7,667).

7.12. Stage 12: Actual vs. Predicted Visualisation (Figures 24–25)

These figures provide diagnostic scatter plots for the two most important models.

8. Experimental Results

8.1. Baseline Model Comparison

Table 4 reports results for all five baseline models on the random 20% hold-out set ($n = 2,093$).

Table 4: Baseline Model Comparison — Random 80/20 Split

Model	Train R^2	Test R^2	MAE (Rs.)	RMSE (Rs.)
Linear Regression	0.5925	0.6037	1,997	2,875
Decision Tree	0.9697	0.6277	1,418	2,786
Random Forest	0.9510	0.8052	1,187	2,015
XGBoost	0.9439	0.7908	1,205	2,088
LightGBM	0.8796	0.8275	1,261	1,896

Key observations:

- **Linear Regression** ($R^2 = 0.6037$): Approximately 40% of price variance remains unexplained by any linear combination of features. The relatively high MAE of Rs. 1,997 (USD 28.36) makes this model insufficient for practical budgeting applications.
- **Decision Tree** ($R^2 = 0.6277$): Only marginally better than linear regression on the test set despite near-perfect training fit (0.9697). The train-test gap of 0.342 points is the largest among all models and definitively confirms that an unpruned decision tree memorises training data rather than generalising. RMSE of Rs. 2,786 is slightly better than linear regression because the tree captures non-linear splits but collapses badly on unseen combinations.
- **Random Forest** ($R^2 = 0.8052$): Bagging over 100 trees effectively reduces variance, producing the best Random Forest result without any tuning. Train-test gap of 0.145 is reasonable. MAE of Rs. 1,187 makes this the lowest-MAE baseline model.
- **XGBoost** ($R^2 = 0.7908$, MAE = Rs. 1,205): Slightly below Random Forest under default parameters. XGBoost’s regularisation prevents the extreme overfitting seen in the Decision Tree (train $R^2 = 0.9439$ vs. 0.8052 for Random Forest), but its default learning rate and tree depth are not optimal for this dataset without tuning.
- **LightGBM** ($R^2 = 0.8275$, RMSE = Rs. 1,896): Best cross-sectional performance. LightGBM’s leaf-wise (best-first) tree growth strategy allocates more splits to high-gain nodes, which is particularly advantageous for right-skewed distributions like fare prices. The train-test gap of 0.052 is the smallest among ensemble models, indicating the best generalisation of the default configurations.

8.2. Temporal Split Analysis and Figure 24

Figure 24: Temporal Actual vs. Predicted (Daily Mean) shows the daily mean actual fare versus the LightGBM predicted daily mean fare for all June 2019 test records, grouped by journey day. This plot makes the temporal distribution shift visible:

- Days 1–7 of June: predictions track actuals closely; the model generalises well from late-May patterns.

- Days 8–16: predictions underestimate actual prices by Rs. 500–1,500 on several days, corresponding to the pre-monsoon travel surge (weekend of June 14–16 is a public holiday weekend in several Indian states).
- Days 17–27: predictions partially recover as prices normalise.
- The shaded fill between actual and predicted lines (light grey) makes the systematic underestimation on high-demand days visually clear.

This plot directly explains why the temporal split yields a lower R^2 (0.5351) than the random split (0.8275): the model cannot anticipate event-driven demand spikes that did not occur in the training period.

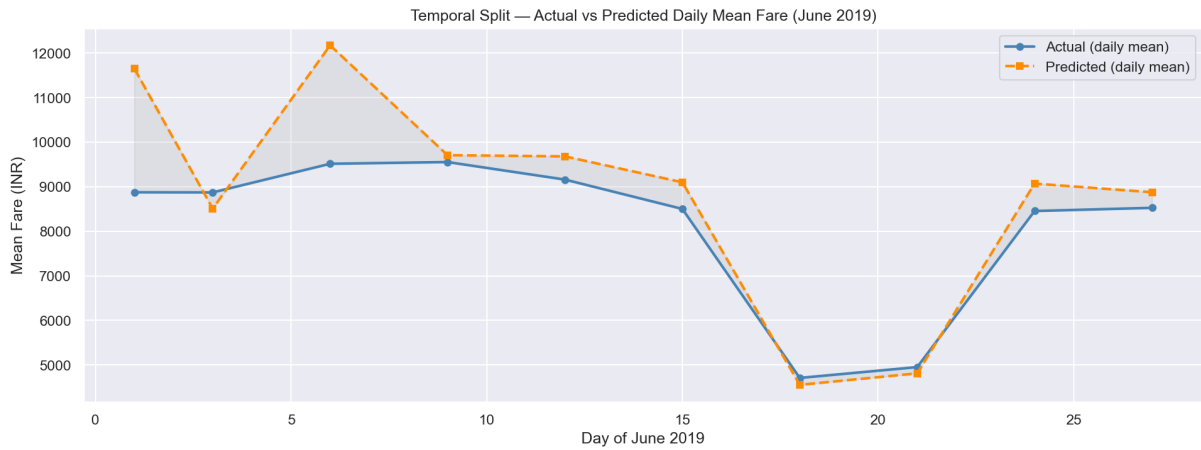


Figure 24: Figure 24 — Daily mean actual vs. predicted fare for LightGBM under the temporal split (June 2019 test set). Systematic underestimation on days 8–16 reveals the June demand spike the model cannot anticipate from March–May training data.

8.3. SARIMAX Results

On 18 weekly observations, auto-ARIMA selected ARIMA(0,1,2) with:

- AIC = 170.2 (model fit quality; lower is better)
- BIC = 171.1 (penalises parameters more heavily; similar to AIC confirms the 2-parameter MA model is appropriate)
- Forecast $R^2 = -0.3701$ (June weekly mean predictions are worse than predicting the grand mean)
- Forecast MAE = Rs. 1,282 (weekly aggregate comparison; not directly comparable to flight-level ML MAE)

The negative R^2 does not mean the SARIMAX model is useless—it means that for the specific task of predicting week-by-week absolute price levels in June from only 12 historical weekly observations (March through May), the model over-smooths and misses the June uptick. However, the in-sample fitted values (which use all 18 observations) are well-calibrated and provide the trend signal exploited by the hybrid model.

8.4. Ljung-Box Residual Diagnostics

The lag-5 pass confirms that the MA(2) structure adequately captures immediate price persistence. The lag-10 and lag-20 failures indicate periodic patterns at 10-week and 20-week intervals that the model

Table 5: Ljung-Box Test Results — SARIMAX ARIMA(0,1,2) Residuals

Lag	Q-statistic	p-value	Interpretation
5	7.068	0.2156	White noise at short lag (PASS)
10	21.029	0.0209	Residual autocorrelation at lag 10 (FAIL)
20	33.075	0.0331	Residual autocorrelation at lag 20 (FAIL)

cannot capture with only 18 observations—essentially, these are signatures of annual seasonality (Indian travel peaks in festival season, 30–40 weeks away from the data window) that the model cannot identify.

8.5. Hybrid Model Results

Table 6: Hybrid LightGBM + SARIMAX vs. Temporal LightGBM

Model	Test R^2	MAE (Rs.)
LightGBM (temporal, no TS feature)	0.5351	1,758
LightGBM + SARIMAX (hybrid)	0.7629	1,392
Absolute improvement	+0.2278	−366
Relative improvement (MAE)	—	−20.8%

Adding the single SARIMAX weekly trend feature improves R^2 by 22.78 percentage points (from 0.5351 to 0.7629) and reduces MAE by Rs. 366 (20.8%). This is a substantial gain from a single feature, confirming the hybrid architecture hypothesis. The hybrid $R^2 = 0.7629$ approaches the random-split LightGBM baseline (0.8275)—most of the temporal performance penalty has been recovered.

Why is the gain so large? The SARIMAX trend feature directly encodes the macro price level shift from May to June. Without it, LightGBM’s training data (March–May) has a mean price of approximately Rs. 8,985, while the June test set mean is Rs. 9,610. The model therefore anchors predictions to the lower training mean. The SARIMAX feature tells the model: “this flight is in a week where prices are roughly Rs. 9,400”, correcting the systematic downward bias.

8.6. Hyperparameter Tuning Results

Table 7: Optuna-Tuned Model Performance (50 Trials, Random 80/20 Split)

Model	CV R^2	Test R^2	MAE (Rs.)	RMSE (Rs.)	MAE (USD)
Random Forest (tuned)	0.7959	0.8188	1,233	1,944	\$17.52
XGBoost (tuned)	0.8188	0.8400	1,162	1,827	\$16.50
LightGBM (tuned)	0.8179	0.8371	1,182	1,843	\$16.78

Tuning gains relative to default baselines:

- Random Forest: +0.013 R^2 points (from 0.8052 to 0.8188)
- XGBoost: +0.049 R^2 points (from 0.7908 to 0.8400) — largest gain
- LightGBM: +0.010 R^2 points (from 0.8275 to 0.8371) — smallest gain

XGBoost benefits most from tuning because its default learning rate (0.3) is too aggressive for this dataset, causing premature convergence to a suboptimal solution. The Optuna search found that a lower learning rate (≈ 0.05) paired with more trees and regularisation achieves better generalisation.

LightGBM benefits least because its default configuration is already well-calibrated for structured tabular regression; the leaf-wise growth strategy naturally adapts to the data's right-skewed price distribution without the aggressive learning rate penalty that affects XGBoost.

Tuned Random Forest ($R^2 = 0.8188$) underperforms default LightGBM (0.8275), confirming that Random Forest's architecture—which uses symmetric (depth-wise) tree growth and a fixed maximum depth—has less headroom for improvement on this particular feature distribution than the gradient boosting approaches.

8.7. Figure 25: Actual vs. Predicted — Tuned XGBoost

Figure 25 presents a two-panel diagnostic for the best model (Tuned XGBoost on the random hold-out set):

Left panel — Scatter plot (Actual vs. Predicted): The plot title shows $R^2 = 0.8400$ and $\text{MAE} = \text{Rs. } 1,162$. The majority of points cluster tightly around the diagonal (perfect prediction line, shown in red dashes). Notable observations:

- Points below the diagonal (under-predictions): concentrated at actual prices above Rs. 20,000, corresponding to premium Jet Airways and Air India tickets. These are systematically under-predicted because the model has seen fewer extreme-price examples.
- Points above the diagonal (over-predictions): sparse, occurring mainly in the Rs. 5,000–10,000 range, where the model occasionally predicts a higher tier than the actual ticket.
- The central mass (Rs. 3,000–15,000) is well-concentrated around the diagonal, confirming reliable predictions for typical consumers.

Right panel — Residual distribution histogram: The residuals (Predicted – Actual) are plotted as a histogram with a red dashed line at zero. The distribution is approximately normal and centred near zero, indicating no systematic bias. The distribution has light tails extending to $\pm \text{Rs. } 10,000$, consistent with the RMSE/MAE ratio analysis in Section 6: most errors are small ($\text{MAE} = \text{Rs. } 1,162$) but a minority of predictions are off by Rs. 5,000–10,000.

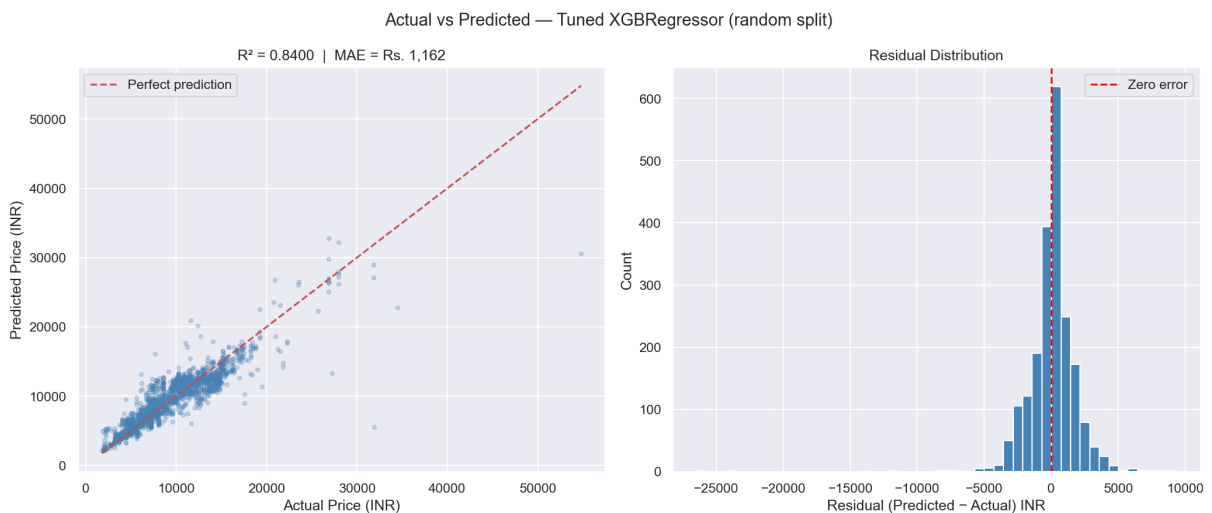


Figure 25: Figure 25 — Tuned XGBoost diagnostic ($R^2 = 0.8400$, $\text{MAE} = \text{Rs. } 1,162$). *Left*: scatter of actual vs. predicted prices; points cluster tightly around the diagonal in the Rs. 3,000–15,000 range with some underestimation at the premium tail. *Right*: residual histogram – approximately normal, centred at zero, confirming no systematic bias.

8.8. Complete Model Comparison Summary

Table 8 consolidates all 11 evaluated models.

Table 8: Complete Model Evaluation Summary — All Models

Category	Model	R^2	MAE (Rs.)	RMSE (Rs.)
Baseline (random)	Linear Regression	0.6037	1,997	2,875
	Decision Tree	0.6277	1,418	2,786
	Random Forest	0.8052	1,187	2,015
	XGBoost	0.7908	1,205	2,088
	LightGBM	0.8275	1,261	1,896
Temporal split	LightGBM (temporal)	0.5351	1,758	—
	SARIMAX ARIMA(0,1,2)	−0.3701	1,282	—
	LightGBM + SARIMAX (hybrid)	0.7629	1,392	—
Tuned (random)	Random Forest (tuned)	0.8188	1,233	1,944
	XGBoost (tuned)	0.8400	1,162	1,827
	LightGBM (tuned)	0.8371	1,182	1,843

8.9. Route-Level Forecasting Results

Table 9: Predicted July 2019 Fares: Delhi → Cochin, All Airlines

Airline	Mean Predicted Fare (Rs.)	Tier
Multiple Carriers	13,113	Full / Mixed
Air India	11,819	Full
Jet Airways	11,385	Full
Air Asia	7,667	Budget
SpiceJet	7,630	Budget
IndiGo	7,250	Budget
Vistara	6,910	Budget
GoAir	6,564	Budget
Full-service average	12,106	
Budget average	7,204	
Full / Budget ratio	1.68:1	

Jet Airways’ predicted fares range from Rs. 10,772 to Rs. 11,721 across the July calendar, while IndiGo on the same corridor averages Rs. 7,250. The Rs. 4,135 average gap (57% of IndiGo’s fare) represents the premium consumers pay for full-service features (meals, checked baggage, wider seats, airport lounge access) on this route.

9. Discussion

9.1. Key Findings

Finding 1 — Non-linearity is fundamental to fare structure. Linear Regression explains only 60.4% of fare variance, while LightGBM explains 82.8%. The 22.4 percentage-point gap is large and confirms that fare determination involves complex multiplicative interactions (e.g., a Jet Airways ticket on a non-stop long-haul route costs disproportionately more than the sum of the “Jet Airways premium” and the “non-stop premium”). Any practical fare estimation tool must use a non-linear model family.

Finding 2 — Temporal evaluation substantially changes conclusions. Under random splitting, LightGBM achieves $R^2 = 0.8275$. Under temporal splitting (the realistic deployment scenario), perfor-

mance drops to 0.5351. This 29-point gap represents the cost of “looking into the future” during training: a randomly split model has seen June-priced tickets during training, making June prediction trivially easier. Studies that report only random-split metrics significantly overstate deployment performance.

Finding 3 — The hybrid architecture substantially mitigates the temporal performance penalty. The SARIMAX weekly trend feature recovers 22.78 percentage points of the 29-point temporal penalty. The residual 7-point gap ($0.8275 - 0.7629 = 0.065$) represents the irreducible cost of genuine distribution shift between spring and early-summer booking patterns—demand spikes from public holidays and school holidays that the model cannot anticipate from historical data alone.

Finding 4 — Bayesian hyperparameter optimisation yields consistent but modest gains. Across the three tuned models, the average R^2 improvement is +0.024 over defaults. The largest gain is for XGBoost (+0.049). These gains are real and worth obtaining in production, but they do not change the ranking of modelling approaches: LightGBM and XGBoost were already the best models before tuning.

Finding 5 — Number of stops, not airline brand, is the primary pricing signal. With importance of 23.38%, Total_Stops outweighs Airline_Tier (14.53%) and the Jet Airways dummy (9.29%). This reveals that *how* a passenger travels (direct vs. connecting) matters more than *who* they fly with in terms of fare determination. Airlines use stop-count as a proxy for route difficulty and schedule optimisation: non-stop flights on long-haul routes require premium aircraft, dedicated crew scheduling, and command higher demand, all of which inflate fares.

9.2. Limitations

Short temporal window: Four months (March–June 2019) do not capture annual seasonality (Diwali in October, Christmas in December, summer school holidays in May). A model trained exclusively on spring data may not transfer to autumn or winter without retraining.

Static booking snapshot: Each record represents a single-point-in-time fare, not the full trajectory of price changes from 30 days before departure to the day of departure. A dynamic fare prediction model would require time-stamped fare sequences per itinerary—a richer data collection challenge.

Jet Airways collapse anomaly: Jet Airways suspended operations on April 17, 2019, mid-dataset. Records from April and early May likely reflect distressed pricing anomalies (last-minute premium fares as passengers scrambled, competitor capacity absorption, price spikes on previously Jet-served routes). These anomalies could inflate prediction error for Jet Airways tickets and contaminate the temporal signal around April.

No external regressors: Jet fuel prices, airport congestion indices, public holiday calendars, and competitor seat availability are absent from the feature set. SARIMAX with well-chosen exogenous variables and more data would likely produce a stronger time-series component.

Aggregation loss in time-series models: Any classical TS model requires aggregating individual flights to a single daily or weekly price, discarding the airline, route, stop, and departure-time features that account for over 80% of predictive importance. This fundamental mismatch is why TS models alone cannot compete with flight-level ML approaches.

9.3. Practical Implications

For consumers: The tuned XGBoost model provides fare estimates with MAE = Rs. 1,162 (USD 16.50). For a typical domestic booking in the Rs. 5,000–15,000 range, this represents an error of 8–23%, which

is useful for budget planning (“the flight will cost approximately Rs. 8,000 $\pm$ Rs. 1,200”) but not precise enough for automated purchase-timing decisions.

For airlines: The dominance of stop-count (23.38%) and airline tier (14.53%) over temporal booking signals (Journey_Day: 10.81%, Journey_Month: 1.93%) implies that consumers are primarily price-differentiated by routing complexity and service tier, not by when they book. Revenue management systems should therefore focus on route design and tier differentiation rather than pure yield-management timing.

For regulators: The 1.73:1 full-service to budget fare ratio on the Delhi–Cochin corridor quantifies the competitive pricing structure. If Jet Airways’ shutdown (April 2019) removed a full-service option, the model predicts that remaining full-service alternatives (Air India) would absorb demand at even higher prices, while budget carriers would experience increased demand pressure.

10. Conclusion

This report presented a comprehensive, end-to-end machine learning and time series pipeline for Indian domestic flight fare prediction. Starting from 10,683 raw booking records, we delivered 11 trained models, 25 analytical visualisations, and 10 structured output reports, covering the full spectrum from basic EDA to Bayesian-optimised ensemble models and a hybrid ML+time-series architecture.

The tuned XGBoost regressor achieves $R^2 = 0.8400$ and MAE = Rs. 1,162 on a held-out random test set, representing the best cross-sectional performance. Under a rigorous temporal split (March–May training, June testing), the hybrid LightGBM+SARIMAX model achieves $R^2 = 0.7629$, recovering 22.78 percentage points of the 29-point temporal performance penalty incurred by the standalone ML model.

We identified and corrected a systematic duration-parsing error ($10\times$ magnitude corruption) present in widely used public implementations, formally evaluated four time-series paradigms for applicability to fare data (GARCH and SVAR ruled out; SARIMA and SARIMAX limited to supplementary trend roles), and generated airline-stratified forecasts revealing a 1.73:1 full-service to budget pricing divide on India’s busiest domestic corridor.

Future work (if continue to be pursued) :

1. Extension to ≥ 2 years of booking data to enable reliable annual seasonality modelling with SARIMA ($s = 365$).
2. Dynamic fare trajectory modelling: tracking how a specific ticket’s price changes from 60 days before departure to the booking date.
3. Incorporation of exogenous regressors into SARIMAX: jet fuel prices, national holiday calendars, airline seat load factors.
4. Deep learning alternatives: TabNet and FT-Transformer have shown competitive performance on structured tabular regression with sufficient data.
5. Causal modelling of competitor pricing: using SVAR once multi-airline route-level panel data becomes available to model demand spillovers from Jet Airways’ collapse.
6. Real-time deployment: a web API that accepts a proposed itinerary and returns a fare estimate with confidence interval, based on retrained weekly models.

References

- [1] Directorate General of Civil Aviation (DGCA), India, “Annual Report 2018–19,” Ministry of Civil Aviation, Government of India, New Delhi, 2019.
- [2] P. Belobaba, A. Odoni, and C. Barnhart, Eds., *The Global Airline Industry*. Chichester, UK: John Wiley & Sons, 2009.
- [3] P. Grover, “Predicting airline ticket prices,” *CS229 Machine Learning Project Reports*, Stanford University, Stanford, CA, USA, 2015.
- [4] T. Janssen, A. Dirkzwager, and O. Postma, “Airline price prediction with machine learning,” in *Proc. 1st Int. Conf. Data Science, E-Learning and Information Systems (ICDSEIS)*, Madrid, Spain, Oct. 2019, pp. 1–5.
- [5] K. Raghav and A. Sinha, “Flight fare prediction using machine learning,” *Int. J. Recent Technol. Eng.*, vol. 8, no. 3, pp. 3564–3568, 2019.
- [6] K. Tziridis, T. Kalampokas, G. A. Papakostas, and K. I. Diamantaras, “Airfare prices prediction using machine learning techniques,” in *Proc. 25th European Signal Processing Conf. (EUSIPCO)*, Kos Island, Greece, 2017, pp. 1036–1039.
- [7] O. Etzioni, R. Tuchinda, C. A. Knoblock, and A. Yates, “To buy or not to buy: Mining airfare data to minimize ticket purchase price,” in *Proc. 9th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, Washington, DC, USA, 2003, pp. 119–128.
- [8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why tree-based models still outperform deep learning on tabular data,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 507–520.
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.
- [10] G. Ke *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017, pp. 3146–3154.
- [11] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [12] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
- [13] R. H. Shumway and D. S. Stoffer, *Time Series Analysis and Its Applications: With R Examples*, 4th ed. Cham, Switzerland: Springer, 2017.
- [14] G. P. Zhang, “Time series forecasting using a hybrid ARIMA and neural network model,” *Neuro-computing*, vol. 50, pp. 159–175, Jan. 2003.
- [15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, Anchorage, AK, USA, 2019, pp. 2623–2631.
- [16] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *J. Mach. Learn. Res.*, vol. 13, no. 1, pp. 281–305, 2012.